

Syrian Private University
Faculty of Artificial Intelligence Engineering
Department of Information Engineering
Information Systems and Computer Networks
Security Engineering



Design and Simulation of an Authentication System in an Internet of Things (IoT) Environment

(A Project Submitted for the Degree of Engineering – Graduation Project I)

Prepared by

Amira Bdier
Thouraya Sheikha

Supervised by

Dr. Wassim Junaidi

Eng. Yamen AL-Hallak

August 2026

Table of Contents

Abstract	8
Introduction	9
Introductory Note on the General Field	9
Introductory Note on the Specific Field	9
Research Problem.....	9
Research Objective	10
Reference Study	11
Light Weight Authentication Scheme for Smart Home IoT Devices	11
Enhancing IEEE 802.1X Authentication with Identity Providers Using EAP-OAUTH and OAuth 2.0	11
Radius Authentication in Wireless Lab Environment	11
LDAP: Framework, Practices, and Trends.....	11
Measurement and Analysis of LDAP Performance.....	11
On Attacking Kerberos Authentication Protocol in Windows Active Directory Servies:A pratical Survey11	
Hybrid Approaches (ABAC and RBAC) Toward Secure access control in Smart Home IOT	11
A Model-DRIVEN Reverse Engineering Approach for Detecting privilege Escalation in IOT System	11
A Single Pass and One Round Message Authentication-Encryption for Limited IoT Devices.....	11
Is Your Kettle Smarter Than a Hacker? A Scalable Tool for Assessing Replay Attack Vulnerabilities on Consumer IoT Devices	11
Perspectives on IoT-Oriented Network Simulation Systems.....	11
Practical Applications	13
Challenges	13
The Internet of Things (IoT)	16
The Concept of the Internet of Things	16
The Evolution of the IoT Concept.....	17
General Characteristics of IoT Networks.....	17
Constrained IoT Environments.....	18
General Security Considerations in IoT	19
Security Challenges in Smart Home IoT	19
Security Requirements in Smart Home IoT	19
Authentication Challenges	19
Authorization Challenges	20
Communication Security Challenges	20
Authentication, Authorization, and Accounting (AAA).....	21
Overview of AAA.....	21

Authentication.....	22
Authorization.....	22
Accounting.....	22
RADIUS	23
Overview of RADIUS	23
RADIUS Authentication Process	23
Advantages of RADIUS in IoT Environments	24
Relevance of RADIUS to the Proposed System	25
LDAP	25
Overview of LDAP	25
LDAP Directory Structure	26
LDAP in Authentication Systems	27
Relevance of LDAP to the Proposed System	27
IEEE 802.1X.....	27
IEEE 802.1X Architecture.....	28
Port-Based Network Access Control.....	28
Authentication States.....	28
Relevance of IEEE 802.1X to the Proposed System.....	29
EAP and EAPOL.....	29
Extensible Authentication Protocol (EAP)	29
Extensible Authentication Protocol over LAN (EAPOL)	29
Relationship Between EAP, EAPOL, and IEEE 802.1X	30
Relevance to the Proposed System	31
Kerberos	31
Kerberos Architecture	32
Ticket-Based Authentication	32
Advantages of Kerberos	33
Relevance to the Proposed System	33
Role-Based Access Control (RBAC)	34
RBAC Model.....	34
Advantages of RBAC in Smart Home IoT	35
RBAC and Privilege Management.....	35
Relevance of RBAC to the Proposed System	36
Message Authentication Code(MAC)	36

Chapter

Chapter Introduction.....	44
Hardware and Software Environment.....	44
Ubuntu and VMware	45
ns-3.41	46
Building and Running the Simulation	47
C++ Implementation.....	48
Network Nodes and Ports	48
Users, Devices, Roles, and Permissions.....	50
Users, Roles, and Permissions	52
IoT Devices and Required Permissions.....	52
Simulation Limitations.....	53
Chapter Summary.....	55

Chapter

Chapter Introduction.....	57
Baseline Architecture Before IEEE 802.1X.....	57
Proposed Architecture After IEEE 802.1X.....	59
System Components.....	60
Supplicant / Principal.....	61
Authenticator	61
RADIUS Server	61
LDAP Server	62
Authentication Workflow	63
Initial Unauthorized State	63
Password Salt and Hash	64

Chapter

Chapter Introduction.....	75
Evaluation Methodology	75
Evaluation Principals and Access Matrix	76
Normal System Results.....	77
IEEE 802.1X Results.....	78
Gateway and RBAC Results	79
LDAP Cache Evaluation.....	80
Without Cache.....	80
With Cache	81

Comparison	81
Replay Attack Evaluation.....	83
Privilege Escalation Evaluation.....	86
Discussion	89

List of Figures

Figure 1	General Conceptual Model of the Internet of Things	16
Figure 2	Authentication, Authorization, and Accounting (AAA) Framework.....	22
Figure 3	Centralized RADIUS Authentication Process.....	24
Figure 4	Example of an LDAP Directory Information Tree	26
Figure 5	IEEE 802.1X Authentication Architecture and EAP/EAPOL Communication.....	31
Figure 6	Kerberos Ticket-Based Authentication Model	33
Figure 7	Role-Based Access Control (RBAC) Model.....	35
Figure 8	Message Authentication Code Generation and Verification	37
Figure 9	Replay Attack Detection Using Sequence Numbers	39
Figure 10	Ubuntu 24.04 LTS Virtual Machine Used for the ns-3.41 Simulation Environment	46
Figure 11	Building the Proposed Simulation in ns-3.41	47
Figure 12	Baseline Security Architecture Before IEEE 802.1X.....	58
Figure 13	Proposed Multilayer Authentication and Access Control Architecture for Smart-Home IoT60	
Figure 14	IEEE 802.1X Authentication Workflow in the Proposed Architecture	66
Figure 15	Gateway Security Enforcement Pipeline.....	70
Figure 16	Normal System Execution and Security Decisions	78
Figure 17	IEEE 802.1X Authentication Results for Authorized and Unauthorized Principals	79
Figure 18	Authentication Results with LDAP Cache Disabled	81
Figure 19	Authentication Results with LDAP Cache Enabled	82
Figure 20	(a)Replay Attack Execution	84
Figure 21	(b)Replay Attack Counters	84
Figure 22	Replay Attack Results with Replay Protection Enabled	85
Figure 23	Privilege Escalation Attack Execution with Protection Disabled.....	87
Figure 24	Privilege Escalation Attack Results with Protection Disabled	87
Figure 25	Privilege Escalation Attack Results with Protection Enabled	88

List of Tables

Table 1	Summary of Previous Studies.....	12
Table 2	Main Characteristics of IoT Environments and Their Security Implications.....	18
Table 3	Main Security Requirements in Smart-Home IoT Environments	21
Table 4	Security Mechanisms Used Against Unauthorized Privilege Modification	41
Table 5	Hardware and Software Environment Used in the Simulation	45
Table 6	Network Nodes Used in the Final Simulation.....	49
Table 7	UDP Ports Used in the Final Simulation	50
Table 8	Principals, Roles, Account Status, and Permissions	51
Table 9	IoT Devices, Supported Commands, and Required Permissions.....	52
Table 10	Functional Modeling of Security Mechanisms	54
Table 11	Main Components of the Proposed Security Architecture	63
Table 12	Security Information Associated with the Kerberos-Inspired Ticket	68
Table 13	Attack Senario and protocol mechanism	76
Table 14	Evaluation Scenarios of the Proposed Architecture.....	77
Table 15	Evaluation Principals and Access-Control Matrix.....	80
Table 16	Normal Gateway and RBAC Command Results	82
Table 17	Comparison of LDAP Authentication Results with Cache Disabled and Enabled.....	85
Table 18	Comparison of Replay Attack Results with Replay Protection Disabled and Enabled	88
Table 19	Comparison of Privilege Escalation Results with Protection Disabled and Enabled	89
Table 20	Overall Evaluation Summary	89

Abstract

The rapid growth of the Internet of Things (IoT) has enabled the development of smart environments where users, sensors, and actuators continuously exchange data and execute control commands. However, the limited computational resources of IoT devices make it challenging to deploy strong security mechanisms directly on end devices, exposing smart-home systems to threats such as unauthorized access, privilege escalation, command manipulation, and replay attacks.

This project presents the design and simulation of a multi-layered security architecture for smart-home IoT environments using ns-3.41. The proposed architecture integrates IEEE 802.1X/EAP for network admission control, RADIUS for authentication, authorization, and accounting (AAA), LDAP for centralized identity and permission management, and a Kerberos-inspired Key Distribution Center (KDC) for issuing secure, time-limited tickets. An independent Authenticator node ensures that every principal remains unauthorized until successful authentication is completed through the IEEE 802.1X/EAP and RADIUS workflow.

After successful authentication, the Gateway acts as the security enforcement point by validating Kerberos tickets, verifying principal binding and ticket expiration, checking a Message Authentication Code (MAC), enforcing Role-Based Access Control (RBAC), validating session identifiers, and detecting replay attempts using sequence numbers. The architecture also incorporates salted password hashing, nonce-based authentication, and an LDAP cache to reduce authentication latency while preserving security.

The proposed system is evaluated through multiple security scenarios, including normal authentication, LDAP cache evaluation, replay attack, and privilege escalation attack. Experimental results demonstrate that the architecture successfully admits only authorized principals, reduces authentication delay through LDAP caching, detects and blocks replayed commands using sequence-number validation, and prevents privilege escalation by verifying command integrity through MAC-based protection. These results confirm that the proposed architecture provides effective layered security while maintaining the lightweight characteristics required for resource-constrained IoT devices.

All protocols and security mechanisms are implemented as functional models within the ns-3.41 simulation environment for research and evaluation purposes rather than full production implementations.

1. Introduction

1.1 Introductory Note on the General Field

Information engineering and computer networks have undergone significant development in recent decades, as computing systems have evolved from isolated and limited environments into large-scale interconnected infrastructures that support continuous communication, distributed processing, and real-time data exchange. This evolution has enabled the emergence of intelligent systems capable of collecting, analyzing, and responding to data generated by a wide range of physical and software components.

As modern systems become increasingly dependent on continuous connectivity, network reliability, scalability, and security have become essential design requirements. Protecting identities, controlling access, preserving message integrity, and preventing unauthorized actions are now fundamental challenges in the development of connected environments. Consequently, network security has become a major area of research within information engineering, particularly in systems that integrate users, servers, sensors, and intelligent devices.

1.2 Introductory Note on the Specific Field

The Internet of Things (IoT) is one of the most significant applications of modern information engineering. It connects physical entities—such as sensors, actuators, smart appliances, and control systems—to communication networks, enabling them to exchange data and perform automated operations with minimal human intervention.

Because many IoT devices operate with limited processing power, memory, and energy resources, security mechanisms must be designed carefully to avoid excessive computational overhead. These limitations make IoT environments particularly vulnerable to unauthorized access, identity misuse, command tampering, replay attacks, and privilege escalation. Therefore, effective IoT security must combine strong authentication, centralized identity and permission management, secure session establishment, and command-level access control while preserving the lightweight nature of constrained devices.

1.3 Research Problem

Although authentication mechanisms are widely used in IoT environments, many smart-home systems still face significant security challenges beyond the initial authentication phase. Ensuring that only legitimate principals gain access to the network is not sufficient on its own. After authentication, the system must continue to verify user identity, enforce access permissions, protect command integrity, and prevent unauthorized or repeated operations throughout the communication session.

In resource-constrained IoT environments, the absence of a comprehensive security architecture may allow unauthorized network access, privilege misuse, replayed valid commands, or forged command requests to reach IoT devices. Furthermore, repeated identity verification may increase

authentication overhead, while weak access-control mechanisms can expose critical smart-home devices to unauthorized operations.

Therefore, there is a need for a lightweight, multi-layered security architecture that combines secure network admission, centralized identity management, ticket-based session control, command integrity verification, replay protection, and role-based authorization while maintaining the efficiency required by constrained IoT devices.

1.4 Research Objective

This project aims to design, implement, and evaluate a multi-layered authentication and access-control architecture for smart-home IoT environments using the ns-3.41 simulation platform. The proposed architecture is intended to ensure that every principal is authenticated before gaining network access, that identities, roles, account states, and permissions are managed centrally, and that only authorized commands are allowed to reach IoT devices.

To achieve this objective, the system functionally integrates IEEE 802.1X/EAP for network admission control, RADIUS for authentication, authorization, and accounting, LDAP for centralized identity and permission management, and a Kerberos-inspired Key Distribution Center for issuing session-bound and time-limited tickets. The architecture also applies Message Authentication Code verification, sequence-number-based replay protection, Role-Based Access Control, session-ID validation, salted password hashing, nonce-based credential proofs, and LDAP caching.

The project further aims to evaluate the effectiveness of the proposed architecture under normal operation and security-attack scenarios, including replay attacks and privilege-escalation attempts. It also investigates the impact of LDAP caching on authentication performance while ensuring that access-control decisions remain unchanged. The overall objective is to provide a structured and lightweight security model that strengthens network admission, session control, and command-level authorization without placing complex security processing directly on resource-constrained IoT devices.

1.5 Reference Study

1.5.1 Light Weight Authentication Scheme for Smart Home IoT Devices

(Vipin Kumar et al., 2022)

1.5.2 Enhancing IEEE 802.1X Authentication with Identity Providers Using EAP-OAUTH and OAuth 2.0

(Duarte Mortágua et al., 2024)

1.5.3 Radius Authentication in Wireless Lab Environment

(Petras Kasiulynas, 2016)

1.5.4 LDAP: Framework, Practices, and Trends

(Vassiliki Koutsonikola and Athena Vakali, 2004)

1.5.5 Measurement and Analysis of LDAP Performance

(Xin Wang et al., 2008)

1.5.6 On Attacking Kerberos Authentication Protocol in Windows Active Directory Services: A practical Survey

(Carlos Díaz Motero et al., 2021)

1.5.7 Hybrid Approaches (ABAC and RBAC) Toward Secure access control in Smart Home IOT

(Safwa Ameer, 2022)

1.5.8 A Model-DRIVEN Reverse Engineering Approach for Detecting privilege Escalation in IOT System

(Manar H. Alalfi et al., 2023)

1.5.9 A Single Pass and One Round Message Authentication-Encryption for Limited IoT Devices

(Hassan N. Noura et al., 2022)

1.5.10 Is Your Kettle Smarter Than a Hacker? A Scalable Tool for Assessing Replay Attack Vulnerabilities on Consumer IoT Devices

(Sara Lazzaro et al., 2024)

1.5.11 Perspectives on IoT-Oriented Network Simulation Systems

(Alberto Gallegos Ramonet et al., 2024)

Table 1 Summary of Previous Studies

#	Study Title	Year	Field	Main Idea	Strengths	Research Gap
1	Light Weight Authentication Scheme for Smart Home IoT Devices	2022	Smart Home IoT Security	Lightweight authentication for smart-home IoT devices.	Inspired the lightweight multi-layer authentication architecture used in our project.	Does not integrate IEEE 802.1X, RADIUS, LDAP, Kerberos, RBAC, or ns-3.
2	Enhancing 802.1X Authentication with Identity Providers Using EAP-OAUTH	2024	IEEE 802.1X /EAP	IEEE 802.1X authentication using EAP and an Authenticator.	Inspired the implementation of the Supplicant–Authenticator–Authentication Server workflow.	Uses OAuth identity providers instead of LDAP and Kerberos.
3	RADIUS Authentication in Wireless Lab Environment	2016	AAA / RADIUS	Practical implementation of RADIUS authentication and authorization.	Used as the basis for the AAA server in our architecture.	Focuses on enterprise WLAN rather than IoT environments.
4	LDAP: Framework, Practices, and Trends	2004	LDAP Directory	LDAP directory structure, authentication, and identity management.	Used to implement centralized user accounts, roles, and permissions.	Does not address smart-home IoT authentication.
5	Measurement and Analysis of LDAP Performance	2008	LDAP Performance	Evaluates LDAP performance and caching.	Inspired the implementation of LDAP Cache to reduce authentication delay.	Focuses only on LDAP performance without IoT integration.
6	On Attacking Kerberos Authentication Protocol in Windows Active Directory Services: A Practical Survey	2021	Kerberos	Ticket-based authentication using Kerberos.	Used to implement Kerberos tickets, session keys, and ticket expiration.	Focuses on Windows Active Directory rather than lightweight IoT systems.
7	Hybrid Approaches (ABAC and RBAC) Toward Secure Access Control in Smart Home IoT	2022	Access Control	RBAC and ABAC for smart-home access control.	Inspired the RBAC permission model used before command execution.	Proposes hybrid access control instead of centralized Gateway RBAC.
8	A Model-Driven Reverse Engineering Approach for Detecting Privilege Escalation in IoT Systems	2023	Privilege Escalation	Detects privilege-escalation vulnerabilities in IoT systems.	Motivated the privilege-escalation protection added to our project.	Detects attacks rather than preventing them during runtime.
9	A Single Pass and One Round Message Authentication-Encryption for Limited IoT Devices	2022	Message Authentication	Lightweight message authentication and integrity protection.	Inspired the use of Command MAC to protect roles and permissions.	Proposes a new cryptographic algorithm instead of integrating MAC within an authentication architecture.
10	Is Your Kettle Smarter Than a Hacker? A Scalable Tool for Assessing Replay Attack Vulnerabilities on Consumer IoT Devices	2024	Replay Attack / IoT	Replay attack analysis on consumer IoT devices.	Inspired the replay-attack scenario and sequence-number protection.	Focuses on vulnerability assessment rather than prevention architecture.
11	Perspectives on IoT-Oriented Network Simulation Systems	2024	IoT Simulation	Reviews IoT simulation platforms and ns-3 capabilities.	Justified the use of ns-3 to implement and evaluate our proposed architecture.	Does not implement a complete authentication and access-control system.

1.6 Practical Applications

The proposed security architecture can be applied to IoT environments that require controlled network access, centralized identity management, secure session establishment, and protection of device commands. Although the project is implemented and evaluated in a smart-home scenario, its layered design can be adapted to other IoT domains with similar security requirements.

The primary application is smart-home IoT, where users and devices interact with smart locks, lighting systems, cameras, sensors, and smart plugs. In this environment, unauthorized access, modified permissions, or replayed commands may lead to critical actions. The proposed architecture reduces these risks by combining IEEE 802.1X/EAP network admission, RADIUS and LDAP-based identity verification, Kerberos-inspired tickets, RBAC authorization, command MAC verification, and sequence-number-based replay protection.

The architecture may also be adapted to Industrial IoT environments, where sensors, controllers, and actuators exchange operational commands. Centralized authentication and role-based authorization can restrict access according to job responsibilities, while ticket validation, command-integrity checks, and replay protection can help prevent unauthorized or repeated control operations.

Another possible application is smart healthcare, where connected monitoring devices and authorized personnel exchange sensitive information and commands. The proposed model can support identity verification, account-status checking, role-based permissions, and controlled access to devices or services. However, a real healthcare deployment would require additional compliance, privacy, safety, and availability controls beyond the scope of this simulation.

The same principles can also support environmental monitoring and distributed sensor networks. Devices can be registered as authenticated principals, assigned limited telemetry permissions, and prevented from executing operations outside their authorized roles. Therefore, the proposed architecture provides a reusable security model for IoT systems that require lightweight devices to operate behind a centralized and multi-layered security infrastructure.

1.7 Challenges

Designing and implementing a secure authentication and access-control architecture for IoT networks presents several technical and security challenges. The first challenge is providing strong security while preserving the lightweight nature of IoT devices, which have limited processing power, memory, and energy resources. Therefore, traditional security mechanisms cannot always be applied directly without introducing excessive computational and communication overhead.

Another challenge is integrating multiple security technologies into a single architecture. The proposed system combines IEEE 802.1X/EAP for network admission, RADIUS for authentication and authorization, LDAP for centralized identity management, Kerberos for

ticket-based authentication, and RBAC for permission enforcement. Ensuring smooth interaction among these components while maintaining system efficiency required careful architectural design.

Protecting the command-execution phase is also a significant challenge. Even after successful authentication, attackers may attempt replay attacks or privilege-escalation attacks by resending captured commands or modifying user roles and permissions. Addressing these threats required implementing sequence-number-based replay protection together with Message Authentication Code (MAC) verification to ensure command integrity and authenticity before execution.

Finally, evaluating such a layered security architecture requires an appropriate simulation environment capable of modeling realistic IoT communication and authentication workflows. For this reason, the proposed system was implemented and evaluated using the ns-3 simulator, allowing different security scenarios to be tested while measuring the effectiveness of the proposed architecture.

Chapter One

Theoretical Study

1.1 The Internet of Things (IoT)

1.1.1 The Concept of the Internet of Things

The Internet of Things (IoT) is a paradigm that extends Internet connectivity beyond traditional computing devices to include a wide range of physical objects capable of sensing, processing, and exchanging information. By integrating embedded sensors, communication technologies, and lightweight computing capabilities, IoT enables everyday objects to interact with each other and with external services without requiring continuous human intervention. This interconnected environment allows devices to collect real-time data, support automated decision-making, and provide intelligent services across various application domains, including healthcare, industrial automation, transportation, and smart homes [1].

The rapid growth of IoT has transformed conventional devices into intelligent network entities that can monitor their surrounding environment and respond to user requests through standardized communication protocols. This capability improves operational efficiency, enables remote monitoring and control, and facilitates seamless interaction among heterogeneous devices connected through the Internet. As the number of connected devices continues to increase, IoT has become a fundamental technology for developing scalable and intelligent cyber-physical systems that require reliable communication and secure access mechanisms [1].

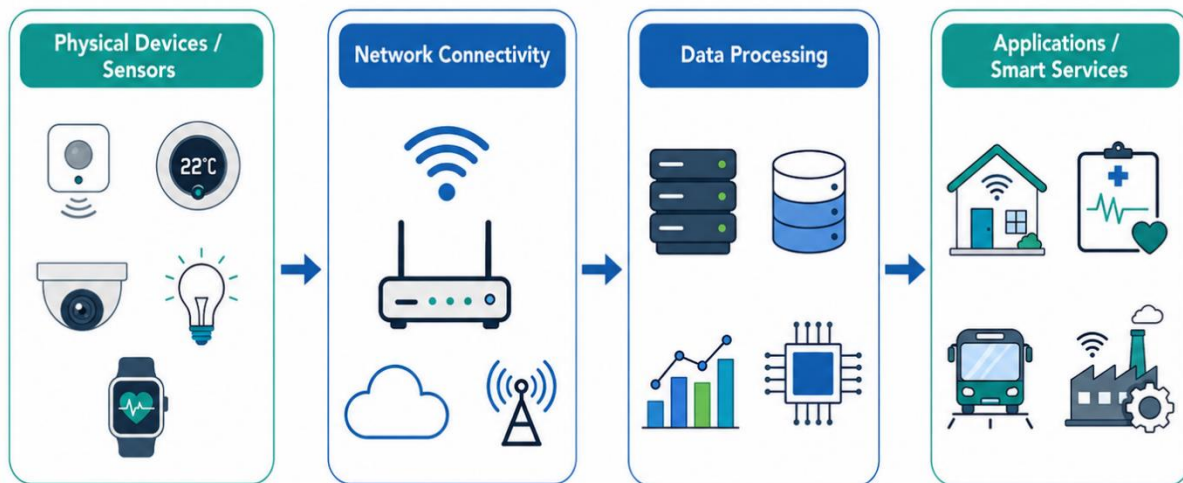


Figure 1 : General Conceptual Model of the Internet of Things

Figure 1: illustrates the general IoT concept in which physical devices collect and exchange data through communication networks to support intelligent applications and services.

This interconnected model enables automated monitoring, control, and decision-making across different IoT environments.

1.1.2 The Evolution of the IoT Concept

The concept of the Internet of Things (IoT) was first introduced in 1999 by Kevin Ashton, who described the use of Radio Frequency Identification (RFID) technology to enable physical objects to be uniquely identified and connected to digital information systems. Since then, rapid advances in wireless communication, embedded systems, low-power sensors, and Internet technologies have significantly expanded the scope of IoT beyond simple object identification. Modern IoT environments now support autonomous communication, real-time data collection, and intelligent interaction among large numbers of interconnected devices without requiring continuous human intervention [1].

As IoT technologies continued to evolve, they became integrated with cloud computing, edge computing, and intelligent data processing techniques, enabling the development of scalable and distributed applications across various domains. Among these applications, smart home environments have emerged as one of the most prominent IoT use cases, where heterogeneous devices cooperate to provide automated services, remote monitoring, and intelligent control. This continuous evolution has also increased the importance of secure authentication and access control mechanisms to protect the growing number of interconnected devices and the sensitive data they exchange [1].

1.1.3 General Characteristics of IoT Networks

IoT networks possess several distinctive characteristics that differentiate them from conventional communication networks. One of their primary features is the ability to interconnect a large number of heterogeneous devices with varying computational capabilities, communication technologies, and functional roles. These devices cooperate to collect environmental data, exchange information, and execute application-specific tasks through continuous network connectivity. Such heterogeneity enables IoT systems to support diverse application domains while introducing additional challenges in communication management and system interoperability [1].

Another important characteristic of IoT networks is the widespread use of resource-constrained devices equipped with limited processing power, memory capacity, and energy resources. Consequently, communication protocols and security mechanisms designed for IoT environments must operate efficiently without imposing excessive computational or communication overhead. These constraints require lightweight and scalable solutions capable of providing reliable connectivity while maintaining acceptable performance for large-scale deployments of interconnected smart devices [1].

1.1.4 Constrained IoT Environments

Most Internet of Things (IoT) deployments operate in constrained environments where devices are designed with limited computational resources, restricted memory capacity, and low energy consumption. These devices are typically connected through lightweight communication technologies and are expected to perform sensing, data processing, and communication tasks while maintaining minimal resource utilization. Such operational constraints distinguish IoT environments from conventional computer networks and require careful consideration during system design [1].

The limited capabilities of IoT devices make the direct adoption of conventional security mechanisms impractical, as many traditional approaches introduce significant computational and communication overhead. Consequently, security solutions for constrained IoT environments should provide effective protection while maintaining low latency, efficient resource utilization, and scalability. These requirements are particularly important in smart home environments, where numerous interconnected devices continuously exchange sensitive information and require secure authentication and controlled access without degrading system performance [1].

Table 2 : Main Characteristics of IoT Environments and Their Security Implications

Security Implication	IoT Characteristic
Lightweight security mechanisms are required	Limited processing power
Security state and stored information should remain compact	Limited memory
Excessive processing and communication should be avoided	Energy constraints
Security mechanisms should support different device types	Device heterogeneity
Authentication and access control should be scalable	Large number of devices
Secure communication and access control are required	Continuous connectivity

Table 2 : summarizes the main characteristics of constrained IoT environments and their implications for security design. These limitations motivate the use of lightweight, scalable, and layered protection mechanisms suitable for resource-constrained smart devices.

1.1.5 General Security Considerations in IoT

Although the Internet of Things offers significant advantages in terms of automation, remote monitoring, and intelligent decision-making, its widespread adoption has introduced important security concerns. IoT devices continuously exchange sensitive information through interconnected networks, making the protection of communication channels, user identities, and

access privileges essential for maintaining system reliability and trustworthiness. The heterogeneous nature of IoT devices and their resource limitations further complicate the implementation of effective security mechanisms, particularly in large-scale deployments such as smart home environments [1], [10].

As IoT ecosystems continue to expand, ensuring secure authentication, controlled authorization, and message integrity has become a fundamental design requirement rather than an optional feature. Consequently, modern IoT systems increasingly rely on layered security architectures that combine multiple protection mechanisms to safeguard both network access and device communications while maintaining efficient system performance [1], [10].

1.2 Security Challenges in Smart Home IoT

1.2.1 Security Requirements in Smart Home IoT

Low-Rate Wireless Personal Area Networks (LR-WPAN) are one of the fundamental pillars for building IoT systems. They were designed specifically to support communication among a large number of resource-constrained devices within a small geographical range. These networks focus on achieving a balance between energy consumption, data-transfer rate, and response time, making them suitable for applications that rely on sensors and smart devices with limited-capacity batteries [2]. Smart home environments rely on the continuous interaction of numerous interconnected IoT devices, including sensors, smart appliances, gateways, and user-controlled applications. These devices exchange operational data and control commands over local networks to provide automation, remote monitoring, and intelligent services. Since many of these communications involve sensitive information and device control operations, maintaining a secure communication environment is essential to ensure system reliability, user privacy, and service availability [1], [10].

The increasing number of connected devices, combined with their heterogeneous characteristics and limited computational resources, introduces significant security challenges. Unauthorized network access, identity impersonation, unauthorized command execution, and message manipulation may compromise the integrity and reliability of smart home systems if appropriate protection mechanisms are not incorporated into the system architecture. Consequently, modern smart home IoT environments require security frameworks that provide strong authentication, controlled authorization, message integrity, and secure communication while remaining suitable for resource-constrained devices [1], [10].

1.2.2 Authentication Challenges

Authentication is the first line of defense in smart home IoT systems, as it ensures that only legitimate users and authorized devices are allowed to join the network and access its resources. However, implementing robust authentication in IoT environments is challenging due to the large number of heterogeneous devices, their limited computational capabilities, and the need for efficient communication with minimal processing overhead. Conventional authentication mechanisms designed for traditional computer networks are often unsuitable for resource-

constrained IoT devices because they may introduce excessive computational complexity and communication latency [1].

Furthermore, smart home environments require authentication mechanisms that can securely verify device identities before granting network access while maintaining scalability as the number of connected devices increases. These requirements have motivated the adoption of lightweight and standardized authentication frameworks capable of providing secure identity verification without significantly affecting system performance. Such mechanisms form the foundation of secure IoT architectures and are essential for preventing unauthorized access to smart home networks [1], [3].

1.2.3 Authorization Challenges

Successful authentication alone does not guarantee secure operation within a smart home environment. After verifying the identity of a user or device, the system must ensure that the requested operations are consistent with the permissions assigned to that entity. Since smart home systems consist of multiple users, devices, and services with different responsibilities, controlling access to protected resources becomes a critical security requirement [7], [8].

Improper authorization policies may allow users or applications to perform operations beyond their intended privileges, increasing the risk of unauthorized device control and misuse of system resources. Therefore, modern smart home security architectures require fine-grained authorization mechanisms capable of associating users with predefined roles and permissions while ensuring that every access request is evaluated before execution. Such an approach minimizes unauthorized operations and strengthens the overall security of IoT environments [7], [8].

1.2.4 Communication Security Challenges

In addition to authentication and authorization, securing communication among IoT devices is essential for preserving the integrity and reliability of smart home systems. Devices continuously exchange authentication messages, control commands, and operational data across the local network. If these communications are not adequately protected, attackers may intercept, modify, or retransmit legitimate messages, potentially leading to unauthorized system behavior and compromising the trustworthiness of the entire IoT environment [1], [9].

The resource limitations of IoT devices further complicate communication security, as protection mechanisms must provide strong security guarantees while maintaining low computational and communication overhead. Consequently, secure smart home architectures increasingly employ complementary protection techniques that verify message authenticity, preserve message integrity, and ensure that transmitted commands remain resistant to common network-based attacks. Integrating multiple security mechanisms across different layers of the communication process provides more comprehensive protection than relying on a single security technique alone [1], [9].

Table 3 : Main Security Requirements in Smart-Home IoT Environments

Purpose	Security Requirement
Verify legitimate users and devices	Authentication
Restrict operations according to permissions	Authorization
Detect unauthorized command modification	Message Integrity
Prevent reuse of previously valid messages	Message Freshness
Maintain protection with limited IoT resources	Efficient Security

Table 3: summarizes the major security requirements identified for smart-home IoT environments. These requirements demonstrate that authentication alone is insufficient and must be complemented by authorization, integrity protection, and message-freshness verification.

1.3 Authentication, Authorization, and Accounting (AAA)

1.3.1 Overview of AAA

Authentication, Authorization, and Accounting (AAA) is a widely adopted security framework that provides centralized control over user identity verification, access management, and activity monitoring in networked systems. Rather than relying on independent security mechanisms, the AAA framework integrates these three complementary functions to ensure that only legitimate users gain network access, authorized entities perform permitted operations, and system activities can be recorded for administrative purposes. This integrated approach has made AAA a fundamental component of modern enterprise networks and IoT environments that require secure and scalable access control [3].

In smart home IoT environments, where multiple users and heterogeneous devices communicate continuously, the AAA framework provides a structured security model capable of managing network access while maintaining centralized control over authentication and authorization processes. Its modular design also enables seamless integration with authentication servers and directory services, making it suitable for secure IoT deployments [3], [4].



Figure 2 : Authentication, Authorization, and Accounting (AAA) Framework

Figure 2 : illustrates the three complementary functions of the AAA framework. Authentication verifies identity, authorization determines permitted actions, and accounting records relevant network and security activities.

1.3.2 Authentication

Authentication is the process of verifying the identity of a user or device before allowing access to network resources. The authentication server validates the credentials presented by the requesting entity and determines whether the claimed identity is legitimate. This process prevents unauthorized devices from participating in network communications and establishes the first layer of defense against unauthorized network access [3].

Within the proposed smart home architecture, authentication serves as the initial security mechanism that verifies user identities before any subsequent authorization or communication procedures are performed. This process establishes the trusted foundation upon which the remaining security components operate, ensuring that only authenticated entities can proceed through the access control workflow.

1.3.3 Authorization

After successful authentication, authorization determines the actions that an authenticated user or device is permitted to perform. Instead of granting unrestricted access, authorization evaluates predefined security policies, user roles, and assigned permissions before allowing access to protected resources. This mechanism enforces the principle of least privilege by ensuring that users can only perform operations consistent with their assigned responsibilities [7].

For smart home IoT systems, authorization is particularly important because different users often require different levels of access to connected devices and services. Applying appropriate authorization policies reduces the possibility of unauthorized device control and enhances the overall security of the network infrastructure [7], [8].

1.3.4 Accounting

Accounting is responsible for recording security-related events associated with authenticated users and network activities. These records may include authentication attempts, resource access, and operational events that support security auditing and administrative monitoring. By maintaining activity logs, accounting assists administrators in tracking system usage, identifying abnormal behavior, and supporting security analysis when required [3].

Although the primary focus of this project is on authentication and authorization, the AAA framework provides the conceptual foundation upon which the proposed security architecture is built. The integration of centralized identity verification, access control, and security management contributes to establishing a reliable and organized security model for smart home IoT environments.

1.4 RADIUS

1.4.1 Overview of RADIUS

Remote Authentication Dial-In User Service (RADIUS) is a client-server protocol designed to provide centralized Authentication, Authorization, and Accounting (AAA) services for network access control. It enables network access devices, acting as RADIUS clients, to forward authentication requests to a centralized RADIUS server, where user credentials are verified before access decisions are made. This centralized approach simplifies security management, improves scalability, and ensures consistent enforcement of authentication policies across network environments [3].

Due to its reliability and interoperability, RADIUS has become one of the most widely adopted protocols for network access control in enterprise and wireless networks. By separating authentication services from access devices, the protocol allows administrators to manage user identities and access policies from a single point, thereby improving both security and administrative efficiency [3].

1.4.2 RADIUS Authentication Process

The RADIUS authentication process begins when a network access device receives authentication credentials from a user or client device. Instead of validating these credentials locally, the access device encapsulates the authentication information within a RADIUS request and forwards it to the RADIUS server. The server then verifies the supplied credentials against an external identity repository or authentication database before generating an authentication decision [3], [4].

Based on the verification result, the RADIUS server returns either an **Access-Accept** message, allowing the user to proceed, or an **Access-Reject** message, denying network access. This centralized decision-making process ensures that authentication policies remain consistent across the network while reducing the security burden on individual access devices [3].

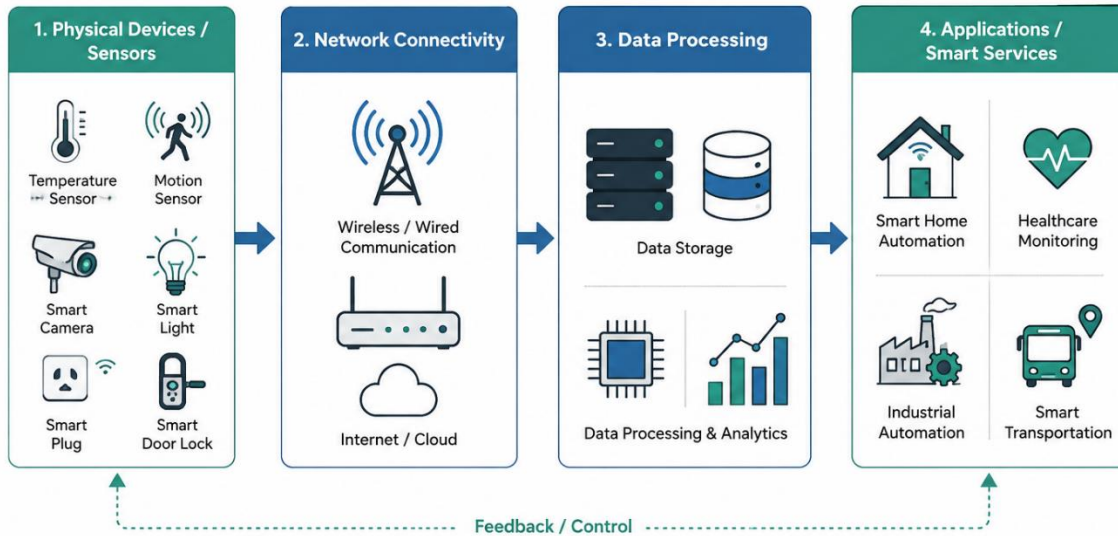


Figure 3: Centralized RADIUS Authentication Process

Figure 3: illustrates the centralized RADIUS authentication process in which authentication requests are forwarded to the RADIUS server for verification using an external identity repository. Based on the verification result, RADIUS returns an Access-Accept or Access-Reject decision to the network access component.

1.4.3 Advantages of RADIUS in IoT Environments

The growing number of interconnected devices in smart home IoT environments requires authentication mechanisms that are both scalable and centrally managed. RADIUS satisfies these requirements by providing a unified authentication framework capable of serving multiple users and devices through a centralized authentication server. This approach simplifies identity management and reduces the complexity associated with maintaining authentication information on individual network devices [3].

Furthermore, centralized authentication improves system maintainability by allowing authentication policies and user information to be updated in a single location. Such an architecture is particularly beneficial for IoT environments, where numerous heterogeneous devices continuously request network access and require consistent security enforcement [3].

1.4.4 Relevance of RADIUS to the Proposed System

In the proposed smart home security architecture, RADIUS functions as the central authentication server responsible for processing authentication requests received from the network authenticator. After verifying user credentials through the directory service, the server generates the appropriate authentication response that determines whether network access is granted or denied. This centralized authentication process establishes the trusted foundation upon which the remaining security mechanisms of the proposed architecture operate.

The integration of RADIUS with the surrounding security components enables the authentication process to be performed in a structured and scalable manner. Within the implemented architecture, the authentication decision produced by the RADIUS server precedes the subsequent authorization and secure communication procedures, thereby ensuring that only authenticated entities are allowed to continue through the remaining security stages of the system.

1.5 LDAP

Lightweight Directory Access Protocol (LDAP) is one of the fundamental technologies used in modern authentication and identity management systems. It provides a centralized directory service for storing and retrieving identity-related information, enabling authentication servers and access control mechanisms to access user data efficiently. Owing to its hierarchical structure and optimized query performance, LDAP has become a widely adopted solution for managing user identities, organizational information, and security attributes in distributed network environments [4], [5].

In the proposed smart home IoT architecture, LDAP serves as the central repository for user accounts, assigned roles, and access permissions. By maintaining identity information in a unified directory, the system ensures consistent authentication decisions while supporting efficient interaction with the authentication server. The following subsections introduce the architecture of LDAP, its directory organization, its role in authentication systems, and its relevance to the proposed security framework [4], [5].

1.5.1 Overview of LDAP

The Lightweight Directory Access Protocol (LDAP) is an application-layer protocol designed to access and manage distributed directory services over Internet Protocol (IP) networks. Unlike conventional relational databases, LDAP directories are optimized for fast read operations and hierarchical organization of identity-related information, making them particularly suitable for authentication and access control applications. Directory entries are organized in a tree-like structure that enables efficient storage and retrieval of information such as user identities, groups, organizational units, and security attributes [4].

Due to its standardized architecture and interoperability, LDAP has become one of the most widely adopted directory service protocols in enterprise and network security environments. It provides a centralized repository that simplifies identity management while allowing multiple network services to retrieve consistent user information from a common directory infrastructure [4].

1.5.2 LDAP Directory Structure

LDAP stores information as a collection of directory entries organized within a hierarchical Directory Information Tree (DIT). Each entry is uniquely identified by a Distinguished Name (DN) and contains a set of attributes describing a specific object, such as a user, device, or organizational unit. This hierarchical organization enables efficient searching, management, and administration of directory information while supporting scalable deployment across distributed environments [4].

The attribute-based structure of LDAP allows identity information to be maintained consistently and updated centrally without requiring modifications across multiple network components. As a result, authentication services and access control systems can retrieve user-related information efficiently while maintaining data consistency throughout the network infrastructure [4].

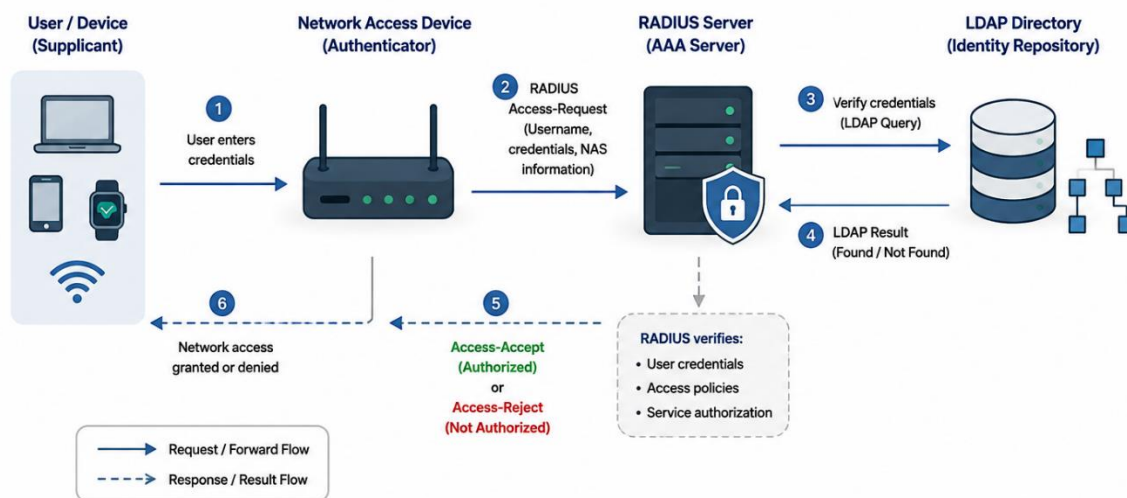


Figure 4: Example of an LDAP Directory Information Tree

Figure 4: presents a conceptual LDAP Directory Information Tree in which identity-related information is organized hierarchically. Such a structure enables centralized storage and efficient retrieval of users, devices, roles, and other authentication-related attributes.

1.5.3 LDAP in Authentication Systems

LDAP is widely employed as the backend directory service for centralized authentication systems. Rather than storing user credentials independently on individual network devices, authentication servers query the LDAP directory to retrieve identity information and verify user accounts. This centralized approach simplifies account administration, improves consistency, and

reduces the complexity associated with managing authentication information across multiple systems [4], [5].

The read-optimized design of LDAP makes it particularly suitable for environments where authentication requests occur frequently. Since directory lookups represent the dominant operation in authentication systems, the protocol provides efficient query performance while supporting large numbers of users and directory objects [5].

1.5.4 Relevance of LDAP to the Proposed System

In the proposed smart home IoT architecture, LDAP functions as the centralized directory service responsible for maintaining user identities together with their associated roles and permissions. During the authentication process, the authentication server retrieves the required identity information from the LDAP directory before making an access decision. Centralizing this information ensures consistency in authentication and authorization while simplifying the administration of users and security policies across the smart home environment.

Since authentication requests may occur repeatedly in IoT environments, frequent directory queries can increase authentication latency. To improve lookup efficiency while preserving centralized identity management, directory caching techniques can be employed to reduce repeated access to the LDAP server. The role of LDAP Cache and its contribution to authentication performance are discussed later in Section **1.10 LDAP Cache**.

1.6 IEEE 802.1X

IEEE 802.1X is a standardized network access control framework developed to provide secure, port-based authentication before permitting access to network resources. Rather than allowing devices to communicate immediately after connecting to a network, IEEE 802.1X requires each requesting entity to complete an authentication procedure through an external authentication server. This approach prevents unauthorized devices from accessing protected network resources and establishes a trusted foundation for subsequent security operations. Owing to its effectiveness and interoperability, IEEE 802.1X has become a widely adopted access control standard in enterprise, wireless, and IoT network environments [2], [3].

In the proposed smart home IoT architecture, IEEE 802.1X represents the first security layer responsible for controlling network admission. It ensures that only authenticated users and devices are permitted to proceed to the subsequent stages of the security framework, thereby strengthening the overall protection of the smart home network [2].

1.6.1 IEEE 802.1X Architecture

The IEEE 802.1X framework is based on a three-entity architecture consisting of the **Supplicant**, the **Authenticator**, and the **Authentication Server**. The Supplicant represents the user or device requesting network access, while the Authenticator acts as an intermediary that

controls access to the network. Authentication decisions are delegated to a centralized Authentication Server, which validates user credentials and determines whether access should be granted or denied [2], [3].

This separation of responsibilities simplifies security management by centralizing authentication decisions while allowing network access devices to focus on controlling access to the communication medium. Consequently, IEEE 802.1X provides a scalable and manageable authentication framework suitable for environments containing numerous interconnected devices, such as smart home IoT networks [2].

1.6.2 Port-Based Network Access Control

A key feature of IEEE 802.1X is its port-based network access control mechanism. Initially, the communication port remains in an unauthorized state, preventing normal network traffic from passing through the Authenticator. During this stage, only authentication-related messages are permitted until the identity of the requesting entity has been successfully verified [2].

Once authentication is completed successfully, the controlled port transitions to an authorized state, allowing the authenticated device to exchange data with other network components. If authentication fails, the port remains blocked, preventing unauthorized devices from accessing network services. This mechanism provides an effective first layer of defense against unauthorized network access [2], [3].

1.6.3 Authentication States

IEEE 802.1X controls network access by maintaining two operational states for each controlled port: an **unauthorized state** and an **authorized state**. When a device initially connects to the network, the controlled port remains in the unauthorized state, allowing only authentication-related traffic while blocking normal data communication. This mechanism ensures that network resources remain inaccessible until the identity of the requesting entity has been successfully verified [2].

Upon successful authentication, the controlled port transitions to the authorized state, permitting normal communication with network services and connected devices. If authentication fails, the port remains unauthorized, preventing the requesting entity from accessing protected network resources. This state-based access control mechanism represents one of the key security features of IEEE 802.1X and provides an effective first layer of defense against unauthorized network access [2], [3].

1.6.4 Relevance of IEEE 802.1X to the Proposed System

Within the proposed smart home security architecture, IEEE 802.1X functions as the initial access control mechanism responsible for regulating network admission. Every user or IoT device must successfully complete the network authentication process before access to protected services is considered. This approach ensures that authentication is performed at the network boundary, preventing unauthorized entities from interacting with internal system components.

The use of IEEE 802.1X also establishes a structured authentication sequence in which network access control precedes centralized identity verification and subsequent authorization procedures. As a result, it provides the foundation upon which the remaining security mechanisms of the proposed architecture operate, contributing to a secure and organized authentication process for smart home IoT environments.

1.7 EAP and EAPOL

The Extensible Authentication Protocol (EAP) and Extensible Authentication Protocol over LAN (EAPOL) are fundamental components of the IEEE 802.1X authentication framework. Together, they provide a standardized mechanism for exchanging authentication information between network entities while allowing different authentication methods to be employed without modifying the underlying access control infrastructure. This flexibility has made EAP and EAPOL widely adopted in enterprise, wireless, and IoT network environments requiring secure and interoperable authentication mechanisms [2].

Within the proposed smart home IoT architecture, EAP and EAPOL facilitate the secure exchange of authentication messages between the requesting device and the network access controller before authentication requests are processed by the centralized authentication server. Their integration with IEEE 802.1X establishes a standardized authentication framework suitable for secure network admission in resource-constrained IoT environments [2], [3].

1.7.1 Extensible Authentication Protocol (EAP)

The Extensible Authentication Protocol (EAP) is a flexible authentication framework that supports multiple authentication methods through a unified message format. Rather than defining a single authentication algorithm, EAP provides a generic framework that enables different authentication techniques to operate within the same protocol architecture. This extensibility allows network administrators to select authentication methods that satisfy the security and operational requirements of their deployment environments [2].

By separating the authentication framework from specific authentication algorithms, EAP promotes interoperability between authentication clients, network access devices, and authentication servers. Consequently, it has become a fundamental component of modern network access control systems that require standardized and scalable authentication procedures [2].

1.7.2 Extensible Authentication Protocol over LAN (EAPOL)

Extensible Authentication Protocol over LAN (EAPOL) is the communication protocol responsible for transporting EAP messages across local area networks between the Supplicant and the Authenticator. Before normal network communication is permitted, EAPOL enables the exchange of authentication messages required to establish the identity of the requesting device.

Once authentication is completed successfully, normal network traffic can proceed through the authorized communication port [2].

Unlike EAP, which defines the authentication message framework, EAPOL specifies how these messages are transmitted over IEEE 802-based local networks. This separation of responsibilities enables IEEE 802.1X to enforce port-based network access control while relying on EAPOL to carry authentication messages between participating entities [2], [3].

1.7.3 Relationship Between EAP, EAPOL, and IEEE 802.1X

IEEE 802.1X, EAP, and EAPOL operate as complementary components within a unified network access control framework. IEEE 802.1X defines the overall authentication architecture and regulates network admission, EAP provides the authentication message framework, and EAPOL transports these messages between the Supplicant and the Authenticator across the local network. Together, these protocols establish a standardized authentication process that supports centralized identity verification through external authentication servers [2], [3].

The coordinated operation of these technologies enables secure and efficient authentication while maintaining interoperability across different network infrastructures. Their standardized design also facilitates integration with centralized authentication systems, making them suitable for smart home IoT deployments requiring scalable and reliable network access control [2].

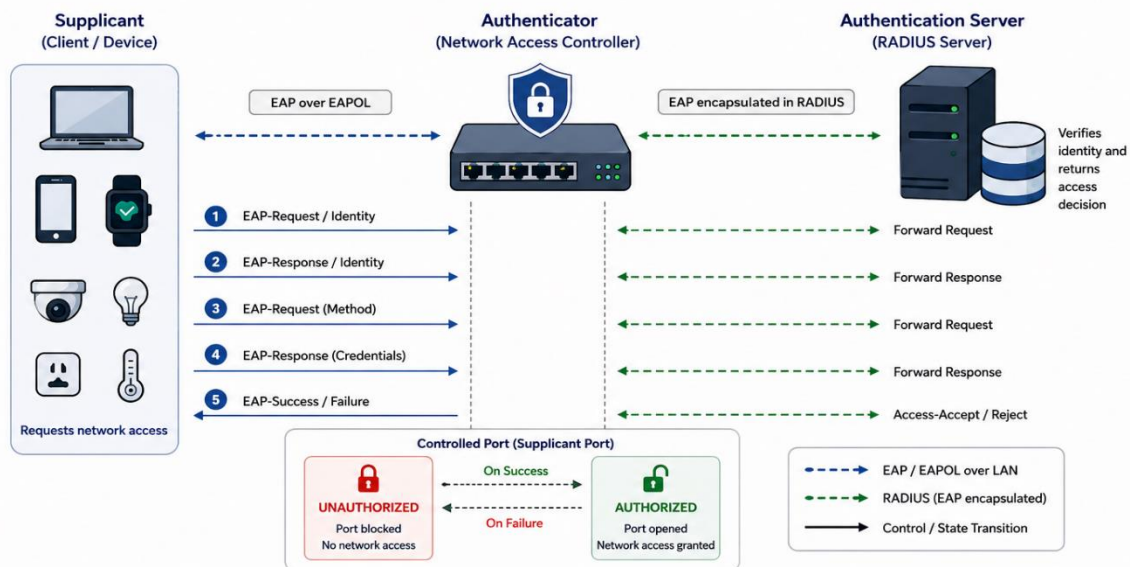


Figure 5 : IEEE 802.1X Authentication Architecture and EAP/EAPOL Communication

Figure 5: illustrates the interaction among the Supplicant, Authenticator, and Authentication Server within the IEEE 802.1X framework. EAP messages are transported over EAPOL between the Supplicant and Authenticator and are forwarded through RADIUS to the Authentication Server, which determines whether network access is authorized.

1.7.4 Relevance to the Proposed System

In the proposed smart home IoT architecture, EAP and EAPOL provide the communication framework used during the initial authentication stage. Authentication messages are exchanged between the Supplicant and the Authenticator using EAPOL before being forwarded to the centralized authentication infrastructure for identity verification. This standardized communication process ensures that authentication is performed consistently while remaining compatible with the IEEE 802.1X access control framework.

By separating authentication message exchange from subsequent authorization and secure communication procedures, EAP and EAPOL contribute to a modular security architecture in which each component performs a dedicated function. This design improves system organization and supports the integration of additional security mechanisms implemented in the proposed architecture without altering the standardized authentication process.

After successful identity verification through the IEEE 802.1X authentication framework, secure service access requires an additional mechanism for managing authenticated sessions. This requirement is addressed through the Kerberos ticket-based authentication model, which is discussed in the following section.

1.8 Kerberos

Kerberos is a centralized network authentication protocol designed to provide secure identity verification over insecure communication networks. Instead of repeatedly transmitting user credentials during service access, Kerberos employs a ticket-based authentication mechanism that enables authenticated users to access network services without exposing their passwords after the initial authentication process. This design reduces the risk associated with credential disclosure while providing a structured authentication framework suitable for distributed computing environments [6].

The protocol has been widely adopted in enterprise infrastructures because it provides mutual authentication between communicating entities and supports centralized authentication management through a trusted authentication authority. These characteristics have made Kerberos an important reference model for designing secure authentication systems that require controlled access to protected resources [6].

1.8.1 Kerberos Architecture

Kerberos is based on a trusted third-party architecture in which a centralized Key Distribution Center (KDC) is responsible for authenticating users and issuing temporary authentication tickets. The KDC consists of two logical components: the Authentication Server (AS), which verifies user identities, and the Ticket Granting Server (TGS), which issues service tickets for accessing protected resources. This architecture enables authentication decisions to be centralized while reducing the repeated exchange of user credentials across the network [6].

By relying on temporary tickets rather than continuous password transmission, Kerberos minimizes credential exposure and provides an organized authentication model suitable for distributed network services. The centralized nature of the KDC also simplifies authentication management and enhances consistency across protected systems [6].

1.8.2 Ticket-Based Authentication

A fundamental feature of Kerberos is its use of temporary authentication tickets. After a user is successfully authenticated by the Authentication Server, the KDC issues a ticket that serves as proof of the authenticated identity when requesting access to network services. This ticket is subsequently presented to authorized services instead of repeatedly transmitting user credentials, thereby improving both security and operational efficiency [6].

The use of temporary tickets reduces the frequency with which sensitive authentication information is exchanged across the network. Consequently, ticket-based authentication has become a widely recognized approach for managing secure access to distributed services while maintaining centralized authentication control [6].

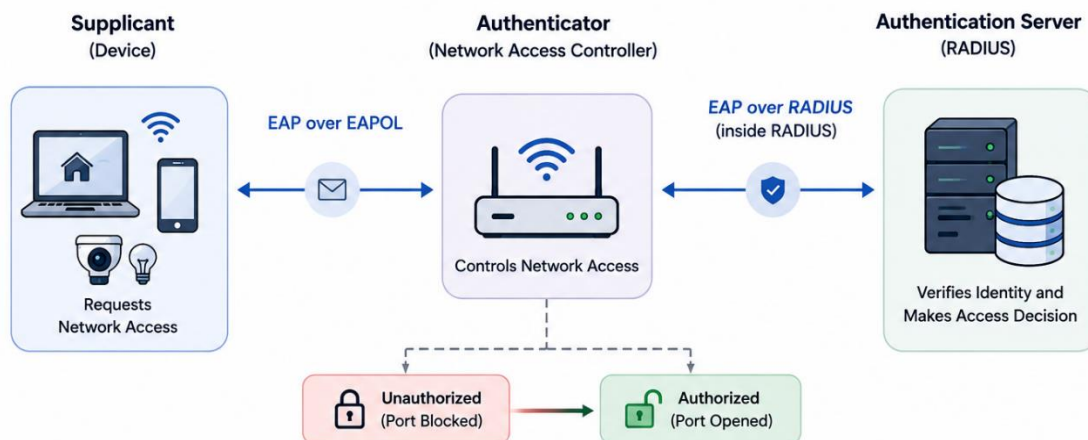


Figure 6 : Kerberos Ticket-Based Authentication Model

Figure 6: illustrates the fundamental ticket-based authentication concept used by Kerberos. Following successful authentication, the KDC issues temporary credentials that can be presented during subsequent access to protected network services instead of repeatedly transmitting the user's original credentials.

1.8.3 Advantages of Kerberos

Kerberos provides several advantages that contribute to secure network authentication. Its centralized authentication model simplifies identity management by delegating authentication decisions to a trusted authority. Furthermore, the use of temporary tickets minimizes repeated credential transmission and supports efficient access to multiple protected services following successful authentication [6].

Another important advantage is the structured separation between user authentication and service access. This design allows authentication to be completed once before temporary credentials are used to access authorized resources, reducing authentication overhead while maintaining a consistent security model across distributed environments [6].

1.8.4 Relevance to the Proposed System

The proposed smart home IoT architecture adopts the fundamental concept of **ticket-based authentication inspired by Kerberos** rather than implementing the complete Kerberos protocol. Following successful user authentication, a centralized component issues a temporary authentication ticket that is subsequently used during the secure access process. This approach reduces the need for repeated identity verification while providing a structured mechanism for managing authenticated access within the system.

Unlike a full Kerberos deployment, the proposed architecture does not implement all protocol components or message exchanges defined by the original specification. Instead, it incorporates the core principle of temporary ticket issuance as part of a multi-layer security architecture that integrates centralized authentication, access control, and secure communication mechanisms. This design provides an authentication model that aligns with the security objectives of the proposed smart home IoT system while remaining compatible with its implementation requirements.

1.9 Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) is one of the most widely adopted authorization models for regulating access to protected resources. Instead of assigning permissions directly to individual users, RBAC associates permissions with predefined roles, while users are assigned one or more roles according to their responsibilities. This approach simplifies access management, improves scalability, and ensures that users perform only the operations permitted by their assigned roles [7].

In smart home IoT environments, where multiple users interact with heterogeneous devices requiring different levels of access, RBAC provides a structured authorization framework that supports centralized policy management and consistent enforcement of access permissions. These characteristics make RBAC particularly suitable for secure IoT systems involving multiple users and interconnected devices [7].

1.9.1 RBAC Model

The RBAC model is based on three fundamental entities: **users**, **roles**, and **permissions**. Users are assigned to one or more roles, while each role is associated with a predefined set of permissions that specify the operations allowed on protected resources. Consequently, authorization decisions are made by evaluating the permissions associated with the user's assigned role rather than the user's identity alone [7].

This abstraction significantly simplifies security administration because changes to access policies are applied by modifying role-permission assignments rather than updating permissions individually for each user. As a result, RBAC improves consistency and reduces administrative complexity in environments with numerous users and protected resources [7].

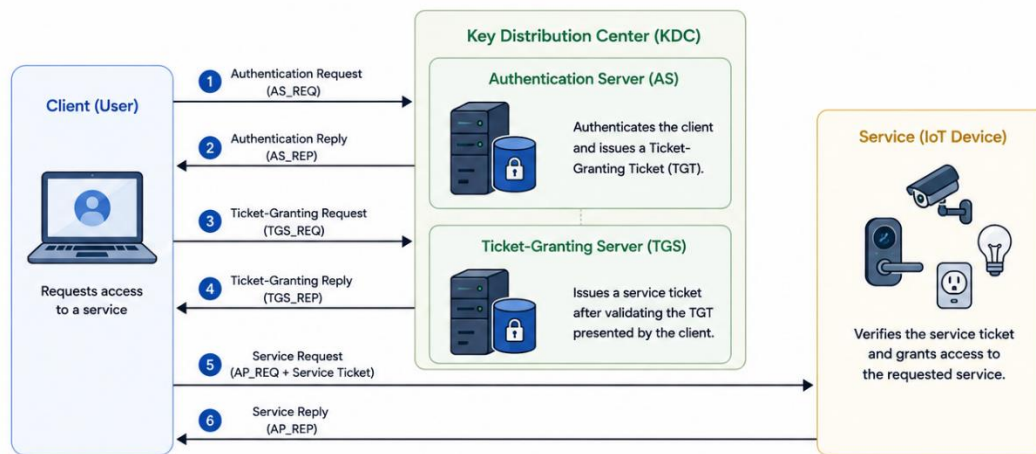


Figure 7 : Role-Based Access Control (RBAC) Model

Figure 7: illustrates the fundamental RBAC relationship among users, roles, and permissions. Users obtain access privileges indirectly through their assigned roles, allowing authorization policies to be managed consistently without assigning permissions independently to every user.

1.9.2 Advantages of RBAC in Smart Home IoT

Smart home IoT environments commonly involve users with different responsibilities, such as homeowners, family members, and guests, each requiring different access privileges. RBAC accommodates these requirements by assigning permissions according to predefined roles instead of individual identities. This approach promotes consistent authorization policies while reducing the likelihood of configuration errors associated with manually assigning permissions to numerous users [7].

Furthermore, centralized role management simplifies the administration of IoT environments as the number of connected devices increases. By separating authentication from authorization, RBAC complements centralized authentication mechanisms while providing fine-grained control over access to smart home services and devices [7].

1.9.3 RBAC and Privilege Management

An essential objective of RBAC is enforcing the principle of least privilege, whereby users receive only the permissions necessary to perform their authorized tasks. Restricting access in this manner reduces the potential impact of unauthorized operations and strengthens the overall security of the system. Proper role assignment therefore plays a significant role in preventing misuse of network resources and protecting connected IoT devices [7], [8].

Effective privilege management also supports the detection and mitigation of abnormal access attempts that exceed the permissions associated with an authenticated user's assigned role. This capability contributes to maintaining consistent authorization policies and protecting critical system resources from unauthorized operations [8].

1.9.4 Relevance of RBAC to the Proposed System

Within the proposed smart home IoT architecture, RBAC provides the authorization mechanism responsible for determining whether an authenticated user is permitted to execute a requested operation. After successful authentication and ticket validation, access decisions are based on the permissions associated with the user's assigned role before any command is forwarded to the target IoT device.

By separating authentication from authorization, the proposed architecture ensures that identity verification alone does not automatically grant unrestricted access to system resources. Instead, authorization decisions are governed by predefined role-permission relationships, thereby supporting secure and organized access control while protecting IoT devices from unauthorized command execution.

1.10 Message Authentication Code(MAC)

A Message Authentication Code (MAC) is a cryptographic mechanism used to verify both the authenticity and integrity of transmitted messages. It enables the receiving entity to confirm that

a message originated from a legitimate sender and that its contents have not been modified during transmission. Unlike encryption, which focuses on protecting the confidentiality of data, a MAC is specifically designed to detect unauthorized modifications while ensuring that the communicating parties share a trusted secret used during the authentication process [9].

In IoT environments, where devices continuously exchange control commands and status information, protecting message integrity is essential for maintaining reliable system operation. Resource-constrained IoT devices particularly benefit from lightweight message authentication mechanisms capable of providing strong integrity protection without introducing excessive computational overhead [9]

1.10.1 Principles of Message Authentication Codes

A Message Authentication Code is generated by applying a cryptographic algorithm to the message together with a shared secret key known only to the communicating entities. The resulting authentication value accompanies the transmitted message and is independently recomputed by the receiver using the same secret key. If both values match, the receiver can verify that the message has not been altered and originated from an authenticated source [9].

Since the MAC depends on both the message contents and the shared secret, any unauthorized modification to the transmitted data results in a different authentication value. Consequently, the receiver can immediately detect message tampering and reject altered or forged communication attempts [9].

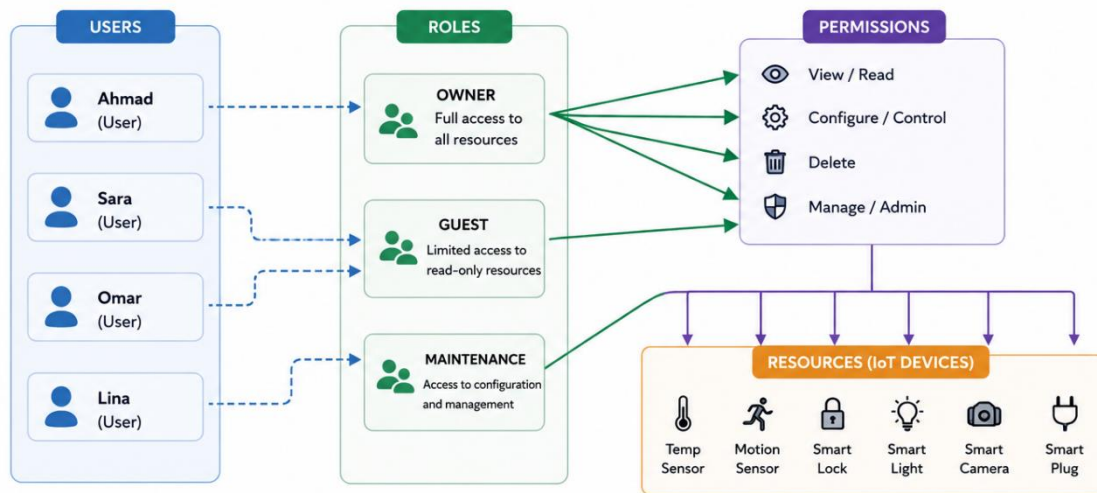


Figure 8 : Message Authentication Code Generation and Verification

Figure 8: illustrates the basic MAC generation and verification process. The receiver recomputes the authentication value using the received message and the shared secret key, allowing unauthorized modifications to be detected when the calculated and received MAC values do not match.

1.10.2 Importance of MAC in IoT Security

IoT systems frequently exchange commands that directly influence the behavior of connected devices. Protecting these commands from unauthorized modification is therefore essential for maintaining secure device operation. Message Authentication Codes provide an effective mechanism for ensuring that transmitted commands remain unchanged while allowing receivers to verify their authenticity before execution [9].

For resource-constrained IoT environments, lightweight MAC mechanisms offer an appropriate balance between computational efficiency and communication security. Their relatively low processing overhead makes them suitable for devices with limited computational capabilities while still providing reliable protection against message tampering [9].

1.10.3 Message Integrity and Authentication

Message integrity and message authentication represent two complementary security objectives achieved through the use of a Message Authentication Code. Integrity verification ensures that transmitted data remains identical to the original message generated by the sender, while authentication confirms that the message originates from an authorized communicating entity possessing the required shared secret [9].

Together, these properties establish a trusted communication channel capable of detecting unauthorized message modification before processing occurs. As a result, MAC mechanisms contribute significantly to maintaining the reliability and security of communications within distributed IoT environments [9].

1.10.4 Relevance of MAC to the Proposed System

Within the proposed smart home IoT architecture, the Message Authentication Code serves as a message integrity verification mechanism applied after successful authentication and authorization. Before a command is forwarded to an IoT device, its authentication value is verified to ensure that the transmitted message has not been modified during communication. This verification contributes to protecting command integrity while preventing unauthorized manipulation of security-related information exchanged within the system.

The use of a MAC complements the preceding authentication and authorization mechanisms by providing an additional layer of protection for transmitted commands. Rather than replacing identity verification or access control, it strengthens the overall security architecture by ensuring that authenticated and authorized commands remain intact before reaching the destination device.

1.11 Replay Attack and Sequence Numbers

The security of IoT systems depends not only on authenticating users but also on ensuring that transmitted messages are fresh and have not been maliciously reused. Even when a device successfully authenticates through secure protocols, an attacker may attempt to capture legitimate communication packets and retransmit them at a later time. This threat is known as a replay attack and represents one of the most common attacks against smart home IoT environments due to the continuous exchange of control commands between users and connected devices [9][1].

1.11.1 Replay Attack

A replay attack is a network-based attack in which an adversary intercepts valid communication messages and retransmits them without modifying their contents. Because the replayed packet contains legitimate authentication information, the receiving device may incorrectly accept it as a new request if no freshness verification mechanism is implemented. In smart home environments, this attack may enable unauthorized execution of previously authorized commands, such as repeatedly unlocking a smart door or activating connected appliances without possessing valid user credentials [9].

Recent studies have shown that replay attacks remain a practical security concern for consumer IoT devices, particularly when communication protocols lack mechanisms for verifying message freshness. As many IoT devices operate under limited computational and energy resources, lightweight replay protection techniques are preferred over computationally intensive security mechanisms [9].

1.11.2 Sequence Numbers

Sequence numbers provide a lightweight mechanism for preventing replay attacks by assigning an incrementing numerical identifier to every transmitted message. The receiving entity maintains the latest accepted sequence number and verifies that each newly received message carries a greater value than the previously processed one. Messages containing duplicated or outdated sequence numbers are immediately rejected, preventing attackers from successfully replaying captured packets [9].

Compared with more complex anti-replay mechanisms, sequence numbers introduce minimal computational overhead while providing effective protection against duplicated message transmission. This makes them particularly suitable for resource-constrained IoT devices where maintaining both security and efficiency is essential [9].

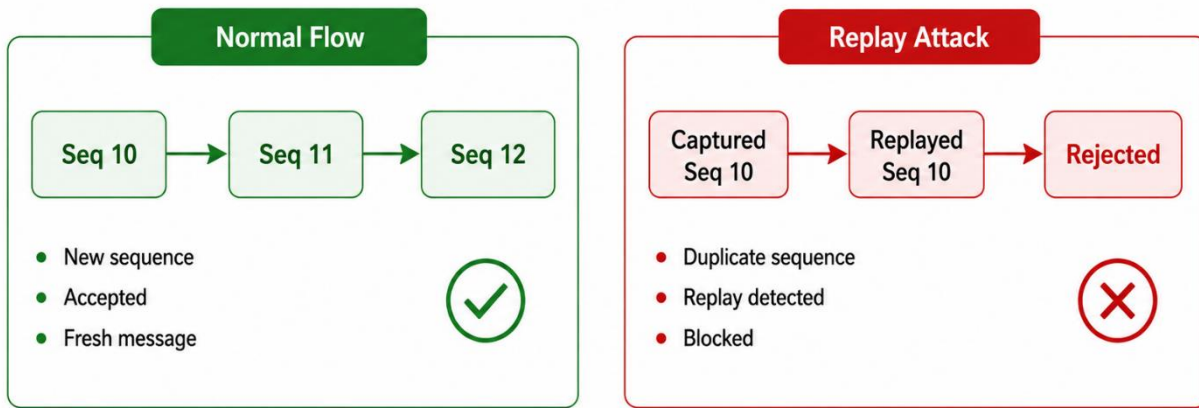


Figure 9 : Replay Attack Detection Using Sequence Numbers

Figure 9: demonstrates how sequence numbers provide message-freshness verification against replay attacks. A duplicated or outdated sequence number identifies a previously processed request and allows the Gateway to reject the replayed message before device execution.

1.11.3 Importance of Sequence Numbers in the Proposed System

In the proposed smart home security architecture, replay protection is implemented after successful authentication and authorization. Each command transmitted from the authenticated user includes a sequence number that is verified by the gateway before forwarding the request to the target IoT device. The gateway compares the received value with the latest accepted sequence number associated with the authenticated session. If the received value is duplicated or lower than the stored value, the command is discarded immediately, preventing replayed messages from reaching the IoT device.

This lightweight verification mechanism complements IEEE 802.1X authentication, RADIUS-based identity verification, Kerberos-inspired ticketing, RBAC authorization, and MAC integrity protection, providing an additional security layer that ensures only fresh and valid commands are executed within the simulated smart home environment [1][9].

1.12 Privilege Escalation

Modern smart home IoT systems support multiple users with different responsibilities and access privileges. Consequently, controlling user permissions has become as important as verifying user identity. Even after successful authentication, a user should only be allowed to perform operations that are explicitly authorized according to the assigned role. Failure to enforce access

control policies may enable unauthorized users to gain elevated privileges and execute restricted operations, resulting in what is known as privilege escalation [8].

1.12.1 Privilege Escalation Attack

Privilege escalation is a security attack in which a legitimate user attempts to obtain permissions beyond those originally assigned by the access control policy. Instead of bypassing authentication, the attacker abuses system weaknesses or improperly enforced authorization rules to access resources or execute commands intended for users with higher privilege levels. In IoT environments, such attacks may compromise device security and violate the principle of least privilege, potentially allowing unauthorized control over critical smart home devices [8].

Research has shown that privilege escalation vulnerabilities frequently arise when authorization mechanisms fail to verify user permissions consistently throughout the command execution process. As IoT systems continue to integrate heterogeneous devices and multiple user roles, implementing reliable authorization policies becomes essential for protecting system resources from unauthorized access [8].

1.12.2 Role-Based Prevention of Privilege Escalation

Role-Based Access Control (RBAC) is one of the most widely adopted approaches for preventing privilege escalation. Rather than assigning permissions directly to individual users, RBAC associates permissions with predefined roles, while users inherit only the permissions assigned to their respective roles. This centralized authorization model simplifies permission management and minimizes the possibility of unauthorized privilege acquisition [7][8].

Within secure smart home environments, RBAC ensures that every requested operation is validated against the user's assigned role before access is granted. Consequently, authenticated users remain restricted to their authorized permissions even after completing the authentication process, providing an effective defense against privilege escalation attacks [7].

1.12.3 Privilege Escalation Protection in the Proposed System

In the proposed smart home security architecture, protection against privilege escalation is achieved through multiple complementary security mechanisms. User roles and permissions are securely maintained within the LDAP directory and retrieved during the authentication process. After successful authentication, the user's assigned role is associated with the issued Kerberos-inspired ticket and subsequently verified by the gateway during every command request.

Before forwarding any command to the target IoT device, the gateway performs RBAC authorization by comparing the requested operation with the permissions assigned to the authenticated user's role. Additionally, the Message Authentication Code (MAC) protects the integrity of the transmitted role and permission information, preventing unauthorized modification during communication. This layered authorization process ensures that authenticated users can execute only the operations explicitly permitted by their assigned roles,

thereby effectively mitigating privilege escalation attacks in the proposed smart home IoT architecture [7][8].

Table 4 : Security Mechanisms Used Against Unauthorized Privilege Modification

Security Function	Mechanism
Stores legitimate roles and permissions	LDAP
Carries authenticated access information	Kerberos-inspired Ticket
Verifies whether the requested operation is permitted	RBAC
Detects modification of command, role, or permission information	MAC
Enforces the final security decision	Gateway

Table 4 : summarizes the complementary mechanisms involved in preventing unauthorized privilege escalation. RBAC verifies whether an operation is permitted, while MAC verification protects the integrity of the role and permission information used during the authorization decision.

1.13 ns-3 Simulation

Network simulation plays an essential role in the design and evaluation of Internet of Things (IoT) systems by providing a flexible environment for analyzing network behavior before real-world deployment. Instead of relying on expensive physical infrastructures, simulation platforms enable researchers to investigate communication protocols, evaluate security mechanisms, and analyze system performance under different network conditions in a controlled and repeatable environment. Consequently, network simulation has become an indispensable tool for the development and validation of secure IoT architectures [10].

1.13.1 Overview of ns-3

ns-3 is an open-source discrete-event network simulator widely used in both academic research and industrial applications. It provides realistic implementations of network protocols and supports the simulation of various communication technologies, making it suitable for evaluating modern networking solutions. Unlike simplified simulation tools, ns-3 allows researchers to implement customized protocol logic using C++, enabling accurate modeling of communication behavior and security-related processes within complex network environments [10].

1.13.2 ns-3 for Smart Home IoT Simulation

The flexibility of ns-3 makes it particularly appropriate for smart home IoT research, where multiple network entities interact through layered communication protocols. Its modular

architecture allows researchers to model authentication servers, gateways, directory services, IoT devices, and communication protocols within a unified simulation environment. This capability enables comprehensive evaluation of authentication workflows, authorization mechanisms, and secure message exchange without requiring physical deployment of smart devices [10].

In this study, **ns-3.41** serves as the simulation platform for implementing the proposed smart home security architecture. The simulator models the interactions among the IEEE 802.1X authenticator, RADIUS server, LDAP directory, Kerberos-inspired ticketing service, gateway, and IoT devices. This simulation environment provides a practical platform for validating the effectiveness of the proposed authentication, authorization, integrity verification, and replay protection mechanisms under controlled experimental conditions.

Chapter Two

Simulation Environment and System Requirements

2.1 Chapter Introduction

The implementation and evaluation of security architectures for smart-home IoT environments require a controlled experimental platform capable of representing multiple users, network entities, authentication services, and IoT devices. Network simulation provides a practical approach for studying such environments because it enables researchers to model communication behavior and evaluate security mechanisms under repeatable conditions without requiring a complete physical deployment of heterogeneous IoT devices [11].

The simulation environment developed in this project is based on **ns-3.41** and C++ and is designed to support the interaction of the security components used throughout the proposed architecture. These components include IEEE 802.1X and EAP/EAPOL for network admission, RADIUS for centralized authentication, LDAP for identity information, Kerberos-inspired ticketing for subsequent protected access, RBAC for authorization, and MAC-based verification for message integrity [2], [3], [4], [6], [7], [9].

This chapter presents the hardware and software environment used to implement the simulation, followed by the Ubuntu and VMware configuration, ns-3.41 platform, C++ implementation environment, network nodes and communication ports, and the users, devices, roles, and permissions represented in the system. It also discusses the limitations of the simulation environment and clarifies the functional protocol modeling approach adopted in the implementation.

2.2 Hardware and Software Environment

The experimental environment was established as a software-based network simulation platform capable of supporting the implementation and evaluation of the proposed smart-home IoT security architecture. Simulation is particularly useful for IoT research because physical testbeds may introduce limitations related to hardware availability, scalability, compatibility, space, and cost, whereas network simulators enable controlled and repeatable evaluation of complex scenarios [11].

The proposed system was implemented on a personal computer using **VMware Workstation** to provide a virtualized Linux environment. **Ubuntu 24.04 LTS** was used as the guest operating system, while **ns-3.41** served as the network simulation framework. The security architecture and simulation logic were implemented in **C++**, and the simulation scenarios were built and executed through the Ubuntu terminal. The use of ns-3 is consistent with its role as a discrete-event network simulator for studying different types of computer networks and IoT-oriented communication scenarios [11].

The hardware platform provides the computational resources required to execute the simulation events and security-processing logic. Processor and memory resources are particularly relevant

when the simulated topology or the number of security operations increases; however, the purpose of this project is to evaluate the behavior of the proposed security architecture rather than to benchmark the physical host computer. The previous project environment likewise identified CPU and RAM as the principal hardware resources used to support ns-3 execution.

Table 5 : Hardware and Software Environment Used in the Simulation

Configuration / Purpose	Component
Personal computer	Host Platform
VMware Workstation	Virtualization Platform
Ubuntu 24.04 LTS	Guest Operating System
ns-3.41	Network Simulator
C++	Programming Language
Ubuntu Terminal	Execution Interface
CPU, RAM, and storage	Main Hardware Resources
Implementation and execution of the smart-home IoT security simulation	Main Purpose

Table 5 : summarizes the main hardware and software components used to implement and execute the proposed simulation environment. The configuration combines a virtualized Ubuntu platform with ns-3.41 and C++ to provide a controlled environment for developing and evaluating the security architecture.

2.3 Ubuntu and VMware

The simulation environment was deployed inside a virtual machine created using **VMware Workstation**. Virtualization provides an isolated workspace in which the operating system, ns-3 installation, compiler, project source files, and simulation dependencies can be maintained separately from the host operating system. In the proposed implementation, VMware therefore serves as the platform hosting the Linux environment in which all simulation activities are performed.

Ubuntu 24.04 LTS was used as the guest operating system for developing, building, and executing the simulation scenarios. The Ubuntu terminal provides direct access to the ns-3.41 project directory, where the C++ simulation files are compiled and executed. This environment also allows the required simulation parameters to be supplied through command-line arguments, enabling different experimental configurations to be tested without changing the underlying operating system environment.

The use of a dedicated Ubuntu virtual machine provides a consistent execution environment for the proposed architecture. All authentication, authorization, attack, and protection scenarios are

executed within the same software platform, which helps maintain consistency between individual simulation runs. The virtual machine is therefore used as the common execution environment for the remaining implementation and evaluation stages described in this study.

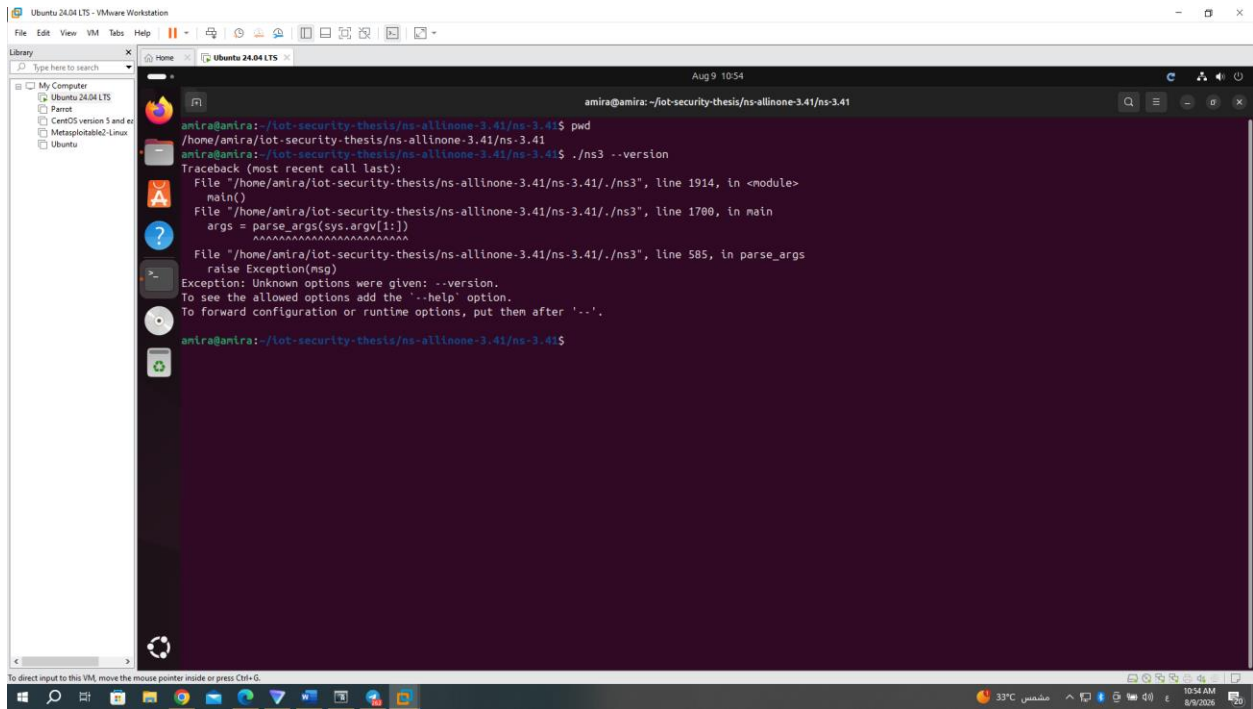


Figure 10 : Ubuntu 24.04 LTS Virtual Machine Used for the ns-3.41 Simulation Environment

Figure 10: presents the Ubuntu 24.04 LTS virtual machine running within VMware Workstation. The terminal displays the ns-3.41 project directory used to develop, build, and execute the proposed smart-home IoT security simulation.

2.4 ns-3.41

The **ns-3 network simulator** is a discrete-event simulation platform designed to model the behavior of different types of computer networks. In discrete-event simulation, system behavior is represented through events occurring at specific simulation times, which makes this approach suitable for studying packet transmission, reception, protocol interactions, and other network-related operations under controlled conditions [11].

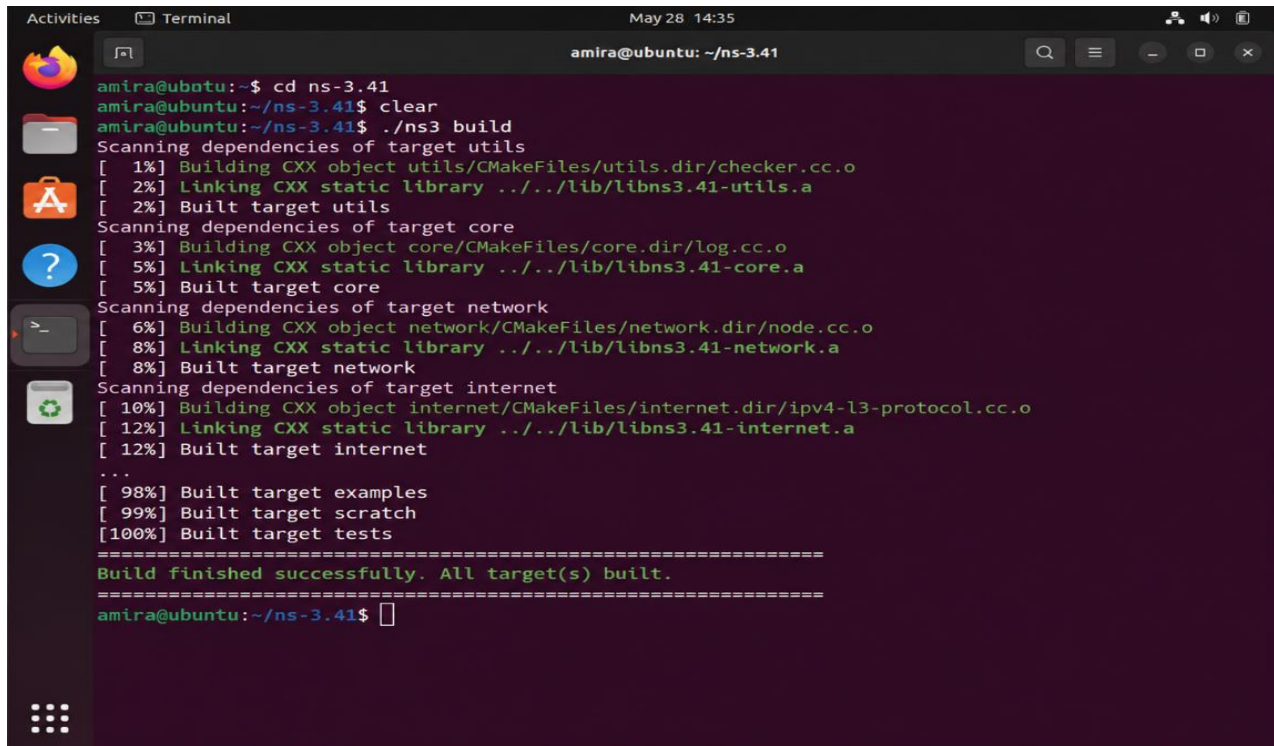
A significant advantage of ns-3 for research-oriented implementations is its programmable and extensible environment. Network scenarios can be constructed through source code, allowing researchers to define network entities, communication behavior, and customized protocol logic according to the requirements of the experimental study. At the same time, this code-oriented

approach requires both programming knowledge and an understanding of computer networking concepts [11].

In this project, **ns-3.41** was selected as the simulation platform for implementing the complete smart-home security environment. The simulator is used to represent the participating users, IoT devices, authentication and directory services, the network Authenticator, the Kerberos-inspired ticketing component, and the security Gateway. The implemented scenarios are executed as discrete simulation events, allowing the authentication workflow, access-control decisions, attack scenarios, and corresponding protection mechanisms to be evaluated within the same controlled environment.

2.4.1 Building and Running the Simulation

After the implementation files are placed within the ns-3 project environment, the simulation must first be built to verify that the source code and required modules can be compiled successfully. The project is then executed through the ns-3 command-line interface, where different simulation parameters can be supplied depending on the scenario being evaluated. This procedure enables the same implementation environment to be reused for normal operation, LDAP cache evaluation, replay-attack testing, and privilege-escalation testing.



```

amira@ubuntu:~$ cd ns-3.41
amira@ubuntu:~/ns-3.41$ clear
amira@ubuntu:~/ns-3.41$ ./ns3 build
Scanning dependencies of target utils
[ 1%] Building CXX object utils/CMakeFiles/utils.dir/checker.cc.o
[ 2%] Linking CXX static library ../../lib/libns3.41-utils.a
[ 2%] Built target utils
Scanning dependencies of target core
[ 3%] Building CXX object core/CMakeFiles/core.dir/log.cc.o
[ 5%] Linking CXX static library ../../lib/libns3.41-core.a
[ 5%] Built target core
Scanning dependencies of target network
[ 6%] Building CXX object network/CMakeFiles/network.dir/node.cc.o
[ 8%] Linking CXX static library ../../lib/libns3.41-network.a
[ 8%] Built target network
Scanning dependencies of target internet
[10%] Building CXX object internet/CMakeFiles/internet.dir/ipv4-l3-protocol.cc.o
[12%] Linking CXX static library ../../lib/libns3.41-internet.a
[12%] Built target internet
...
[ 98%] Built target examples
[ 99%] Built target scratch
[100%] Built target tests
=====
Build finished successfully. All target(s) built.
=====
amira@ubuntu:~/ns-3.41$

```

Figure 11 : Building the Proposed Simulation in ns-3.41

Figure 11: shows the build process of the proposed simulation within the ns-3.41 environment. Successful completion of this process confirms that the implemented C++ simulation files are compiled and ready for execution

2.5 C++ Implementation

The proposed simulation was implemented using C++, which is used in ns-3 for defining network scenarios and customizing simulation behavior. The code-based nature of ns-3 provides flexibility for constructing network entities and implementing specialized application logic, although it requires both programming and networking knowledge [11].

The final implementation is contained in the main source file `scratch/iot-defense-demo.cc` within the ns-3.41 environment. The file integrates the complete security workflow, including IEEE 802.1X/EAP/EAPOL, RADIUS authentication and accounting, LDAP and LDAP Cache, Kerberos-inspired ticketing, Gateway security enforcement, RBAC, Command MAC verification, sequence-number replay protection, and the two implemented attack scenarios.

The implementation is organized into dedicated C++ application components representing the principal security entities of the architecture. Separate application logic is used for the IEEE 802.1X Authenticator, RADIUS server, LDAP server, Kerberos KDC, Smart-Home Gateway, users and sensors, and the attack behavior. This separation allows each security function to be represented independently while still participating in the complete simulated authentication and command-processing workflow.

At the application layer, the C++ implementation models the exchange of protocol-specific information between the simulated entities. EAP messages are transported inside EAPOL between the Supplicant and Authenticator and are then encapsulated inside RADIUS messages for communication with the RADIUS server. Following successful authentication, subsequent ticket and command-processing operations are performed by the dedicated KDC and Gateway components.

2.6 Network Nodes and Ports

The simulated smart-home environment consists of multiple network entities that perform authentication, authorization, directory, ticketing, security-enforcement, user, and IoT-device functions. Since ns-3 is a discrete-event network simulator, the interaction among these entities can be represented as scheduled communication events within a controlled and repeatable simulation environment [11].

In the final implementation, the topology contains **17 ns-3 nodes** connected through a single CSMA network segment. The channel is configured with a data rate of **10 Mbps** and a delay of **2 ms**. An IPv4 protocol stack is installed on all nodes using the `10.1.1.0/24` network. These settings provide the common communication environment required for the authentication, ticketing, command-processing, attack, and protection scenarios implemented in the project.

2.6.1 Network Nodes

The nodes are separated according to their responsibilities within the proposed architecture. Dedicated nodes are assigned to the IEEE 802.1X Authenticator, RADIUS server, LDAP server, Kerberos-inspired KDC, and Smart-Home Gateway. Other nodes represent the human users, sensors, smart-home devices, Replay Attack component, and the isolated Cache Probe used during LDAP Cache evaluation. The final implementation defines exactly **17 nodes**, as summarized in Table 2.2.

Table 6 : Network Nodes Used in the Final Simulation

Main Function	Entity	Node Index
Controls network admission and relays EAP authentication	IEEE 802.1X Authenticator	N_AUTH
Authentication, authorization, and accounting	RADIUS Server	N_RADIUS
Stores identity, role, permission, and credential information	LDAP Server	N_LDAP
Issues tickets to successfully authorized principals	Kerberos-inspired KDC	N_KDC
Central security enforcement point	Smart-Home Gateway	N_GW
Executes authorized door commands	Smart Door Lock	N_DOOR
Executes authorized lighting commands	Smart Light	N_LIGHT
Executes authorized plug commands	Smart Plug	N_PLUG
Generates temperature telemetry	Temperature Sensor	N_TEMP
Generates motion telemetry	Motion Sensor	N_MOTION
OWNER principal	Ahmad	N_AHMAD
GUEST principal	Sara	N_SARA
MAINTENANCE principal	Omar	N_OMAR
DISABLED principal	Lina	N_LINA
Replays a captured command during the replay scenario	Replay Attack Node	N_REPLAY
Generates isolated requests for cache evaluation	LDAP Cache Probe	N_CACHE_PROBE
Executes authorized camera-view requests	Smart Camera	N_CAMERA

Table 6 : summarizes the network entities represented in the final ns-3.41 simulation. The topology separates infrastructure services, users, IoT devices, and evaluation-specific nodes so that each component performs a clearly defined function within the simulated security environment.

2.6.2 Communication Ports

Communication among the simulated applications is organized using predefined **UDP port numbers**. Each major service is assigned a dedicated port so that authentication, accounting, directory lookup, ticketing, Gateway processing, and IoT-device communication can be directed to the appropriate simulated application. The port assignments used in the final implementation are presented in Table 2.3.

Table 7 : UDP Ports Used in the Final Simulation

Purpose	Port	Service
Communication between the Supplicant and IEEE 802.1X Authenticator	8880	EAPOL
Authentication and authorization communication	1812	RADIUS Authentication
RADIUS accounting communication	1813	RADIUS Accounting
Directory lookup communication	3890	LDAP
Ticket request and response communication	8888	Kerberos-inspired KDC
Reception and security processing of protected commands	9000	Smart-Home Gateway
Delivery of approved commands to the target IoT device	9100	IoT Device

Table 7 : presents the UDP port assignments used by the principal communication services in the simulation. Dedicated port numbers allow each ns-3 application to receive and process the messages associated with its specific security or device function.

2.7 Users, Devices, Roles, and Permissions

Role-Based Access Control is used in the proposed architecture to separate user identity from authorization privileges. In an RBAC model, permissions are associated with predefined roles, and users obtain access rights according to the role assigned to them. This approach is

particularly relevant to smart-home IoT environments, where different users may require different levels of access to connected devices and services [7].

In the implemented simulation, the LDAP directory maintains the security attributes required for authorization, including the principal identity, assigned role, account status, and permission mask. Five permission categories are defined: **TELEMETRY, DOOR, LIGHT, CAMERA, and PLUG**. These permissions are subsequently used by the Gateway when determining whether an authenticated principal is authorized to execute a requested operation. The implementation defines the roles **OWNER, GUEST, MAINTENANCE, DISABLED, and DEVICE**.

The principal set consists of four human users and two authenticated IoT sensors. Ahmad is assigned the OWNER role and receives all five implemented permissions. Sara is assigned the GUEST role and is restricted to the LIGHT permission. Omar is assigned the MAINTENANCE role with TELEMETRY and PLUG permissions. Lina is represented as a disabled principal; her account is disabled and no permissions are assigned. The Temperature Sensor and Motion Sensor are represented as DEVICE principals and are restricted to TELEMETRY operations.

Table 8 : Principals, Roles, Account Status, and Permissions

Assigned Permissions	Account Status	Role	Type	ID	Principal
TELEMETRY, DOOR, LIGHT, CAMERA, PLUG	Enabled	OWNER	Human User	100	Ahmad
LIGHT	Enabled	GUEST	Human User	101	Sara
TELEMETRY, PLUG	Enabled	MAINTENANCE	Human User	102	Omar
None	Disabled	DISABLED	Human User	103	Lina
TELEMETRY	Enabled	DEVICE	IoT Sensor	1	Temperature Sensor
TELEMETRY	Enabled	DEVICE	IoT Sensor	2	Motion Sensor

Table 8 : summarizes the principals represented in the authentication and authorization process. Each principal is associated with a predefined role and permission set, while the account-status attribute determines whether the principal is eligible to proceed through the authentication process.

2.7.1 Users, Roles, and Permissions

Role-Based Access Control (RBAC) is used in the proposed smart-home architecture to regulate access to IoT resources according to predefined user roles and their associated permissions. In RBAC-based systems, permissions are assigned to roles rather than directly to individual users, enabling authorization decisions to be applied consistently according to the responsibilities assigned to each user [7].

In the implemented simulation, four human users are defined with different authorization levels. Ahmad is assigned the **OWNER** role and is permitted to access TELEMETRY, DOOR, LIGHT, CAMERA, and PLUG functions. Sara is assigned the **GUEST** role and is restricted to the LIGHT permission, while Omar is assigned the **MAINTENANCE** role with TELEMETRY and PLUG permissions. Lina is represented as a **DISABLED** user and therefore has no assigned permissions. In addition, the Temperature Sensor and Motion Sensor are represented as DEVICE principals with TELEMETRY permission. These assignments are stored in the simulated LDAP directory and are subsequently used by the Gateway when enforcing access-control decisions.

2.7.2 IoT Devices and Required Permissions

In smart-home IoT environments, connected devices provide different services and consequently require different access privileges. Role-Based Access Control (RBAC) provides a structured mechanism for associating permissions with predefined roles, thereby restricting access to IoT resources according to the authorization level assigned to each user [7].

In the implemented simulation, six IoT devices are represented: a Temperature Sensor, Motion Sensor, Smart Door Lock, Smart Light, Smart Camera, and Smart Plug. Each supported operation is associated with a specific permission that must be satisfied before the Smart-Home Gateway forwards the corresponding command to the target device. Temperature and motion reporting require the TELEMETRY permission, while door, light, camera, and plug operations require the DOOR, LIGHT, CAMERA, and PLUG permissions, respectively.

Table 9 : IoT Devices, Supported Commands, and Required Permissions

IoT Device	Device ID	Supported Command	Required Permission
Temperature Sensor	1	SEND_TEMPERATURE	TELEMETRY
Motion Sensor	2	SEND_MOTION	TELEMETRY
Smart Door Lock	3	OPEN_DOOR	DOOR
Smart Light	4	LIGHT_ON	LIGHT
Smart Camera	5	CAMERA_VIEW	CAMERA
Smart Plug	6	PLUG_ON	PLUG

Table 9 : presents the relationship between the simulated IoT devices, the commands supported by the implementation, and the permissions required for each operation. This mapping forms the basis of the RBAC decision performed by the Smart-Home Gateway before a command is delivered to its target device.

2.8 Simulation Limitations

Although network simulation provides a controlled and repeatable environment for evaluating communication systems, it cannot reproduce every characteristic of a real-world deployment with complete fidelity. The accuracy of a simulation depends on the level of abstraction, the implemented models, and the assumptions adopted during scenario design. Open-source IoT simulation platforms may also face limitations related to protocol support, interoperability, documentation, and modeling capabilities; therefore, simulation results should be interpreted within the scope and assumptions of the implemented model rather than as an exact reproduction of a physical system [11].

In this project, the security mechanisms are modeled functionally at the ns-3 application layer rather than implemented as complete production deployments of their corresponding real-world protocols. IEEE 802.1X/EAP, RADIUS, LDAP, Kerberos-inspired ticketing, RBAC, Message Authentication Code verification, and sequence-number-based replay protection are represented through dedicated application logic that reproduces the security decisions and message-processing behavior required by the proposed architecture. In addition, the implemented hash and MAC functions are lightweight simulation models intended for controlled experimental evaluation and should not be interpreted as production-grade cryptographic algorithms.

The experimental evaluation is therefore limited to the scenarios explicitly implemented within the proposed ns-3.41 model. In particular, the security analysis focuses on the normal authentication and authorization workflow, LDAP Cache behavior, Replay Attack detection and prevention, and Privilege Escalation Attack detection and blocking. Consequently, the reported results should be interpreted as an evaluation of the modeled security architecture under controlled simulation conditions rather than as a direct measurement of a complete physical smart-home deployment.

2.9 Functional Protocol Modeling

The proposed system is implemented in ns-3.41 using a functional protocol-modeling approach. Rather than reproducing complete production protocol stacks, the simulation represents the principal security functions and interactions required by the proposed smart-home architecture. This level of abstraction is suitable for controlled network-simulation studies in which protocol behavior and system interactions are evaluated through software models under predefined experimental conditions [11].

Within the implemented environment, IEEE 802.1X/EAP/EAPOL represents network admission, RADIUS provides centralized authentication, authorization, and accounting, LDAP supplies identity and authorization-related information, and Kerberos-inspired ticketing provides temporary access credentials. At the command-processing stage, the Smart-Home Gateway applies ticket validation, Message Authentication Code verification, sequence-number checking, and Role-Based Access Control before an approved request is forwarded to the corresponding IoT device. These mechanisms are functionally integrated at the ns-3 application layer to reproduce the complete security logic of the proposed system.

LDAP Cache is additionally modeled as an optional performance mechanism within the RADIUS authentication process. Previous research has shown that caching can improve LDAP performance when directory information is relatively stable and repeated searches occur [5]. In the proposed simulation, previously retrieved directory information may therefore be reused to reduce repeated LDAP queries while maintaining credential verification during authentication.

Table 10 : Functional Modeling of Security Mechanisms

Security Mechanism	Functional Role in the Simulation	Main Simulated Entity
IEEE 802.1X	Controls network admission and maintains the AUTHORIZED / UNAUTHORIZED state	Authenticator
EAP / EAPOL	Carries authentication information between the Supplicant and the Authenticator	Supplicant / Authenticator
RADIUS	Performs centralized authentication, authorization, and accounting	RADIUS Server
LDAP	Provides identity, account status, role, permission, and credential information	LDAP Server
LDAP Cache	Reduces repeated LDAP queries during authentication	RADIUS Server
Kerberos-inspired Ticketing	Issues a temporary access ticket after successful authorization	KDC
Message Authentication Code	Verifies the integrity of command and privilege information	Smart-Home Gateway
Sequence Number	Detects duplicate commands and protects against Replay Attacks	Smart-Home Gateway
RBAC	Verifies that the authenticated principal has the permission required for the requested operation	Smart-Home Gateway
IoT Command Enforcement	Forwards a command only after all required security checks succeed	Smart-Home Gateway

Table 10 : summarizes the functional role assigned to each security mechanism within the proposed ns-3.41 simulation. Together, these mechanisms form complementary security layers that support network admission, authentication, identity management, ticket-based access, command integrity, replay protection, and role-based authorization before commands are delivered to the simulated IoT devices.

2.10 Chapter Summary

This chapter presented the simulation environment and system requirements used to implement the proposed smart-home IoT security architecture. It described the hardware and software environment, Ubuntu and VMware platform, ns-3.41 simulator, C++ implementation, network nodes and communication ports, as well as the configuration of users, IoT devices, roles, and permissions. The chapter also clarified the simulation limitations and the functional protocol-modeling approach adopted to represent the integrated security mechanisms.

These elements establish the experimental foundation required for implementing and evaluating the proposed system. The next chapter presents the proposed security architecture and implementation in greater detail, including the system components, authentication and authorization workflow, Kerberos-inspired ticketing, Smart-Home Gateway security enforcement, and the implemented protection mechanisms.

Chapter Three

Proposed Security Architecture and Implementation

3.1 Chapter Introduction

Smart-home Internet of Things environments consist of heterogeneous users, devices, communication components, and services that must interact while maintaining secure and controlled access. Because many IoT devices operate with limited processing, memory, and energy resources, security mechanisms must provide effective authentication and access control without imposing unnecessary complexity on endpoint devices [1].

This chapter presents the proposed multilayer authentication and access-control architecture developed for the smart-home IoT environment. The architecture integrates **IEEE 802.1X, EAP/EAPOL, RADIUS, LDAP, LDAP caching, Kerberos-inspired ticketing, Role-Based Access Control (RBAC), Message Authentication Code (MAC) verification, and sequence-number-based replay protection** within the ns-3.41 simulation environment. Each mechanism performs a specific security function within the overall workflow, beginning with network admission and continuing through identity verification, authorization, command-integrity validation, freshness checking, and final command execution.

The proposed architecture follows a defense-in-depth approach in which security decisions are distributed across multiple complementary layers. IEEE 802.1X controls initial network admission, RADIUS provides centralized authentication, LDAP maintains identity and authorization-related information, and the Kerberos-inspired mechanism provides temporary ticket-based access. At the command-processing stage, the Gateway performs RBAC authorization, MAC verification, and sequence-number validation before forwarding an accepted request to the target IoT device. This separation ensures that successful authentication alone does not automatically provide unrestricted access to smart-home resources.

The entire architecture is functionally modeled in ns-3.41 to provide a controlled and repeatable environment for evaluating the interaction between the proposed security mechanisms. Network simulation is particularly useful for IoT research because it enables protocols and security architectures to be evaluated without requiring a complete physical deployment of heterogeneous devices [11].

3.2 Baseline Architecture Before IEEE 802.1X

Before integrating IEEE 802.1X, the baseline architecture relied on centralized authentication and subsequent service-level security mechanisms. In this configuration, the principal initiated the authentication process directly with the RADIUS authentication server. RADIUS then consulted the LDAP directory to retrieve the identity-related information required to evaluate the authentication request.

Centralized RADIUS authentication provides an important advantage by consolidating authentication decisions within a dedicated server rather than requiring individual network components to maintain independent authentication databases. Previous implementations of

RADIUS-based access control have demonstrated the benefits of centralized authentication and centralized user-management mechanisms [3].

After receiving the required LDAP information, the RADIUS server generated either an **Access-Accept** or an **Access-Reject** decision. A successful authentication allowed the principal to continue to the Kerberos-inspired ticketing stage. The principal could then request a temporary ticket from the KDC and subsequently send protected IoT commands through the Gateway. The Gateway performed the required authorization and security checks before forwarding accepted commands to the target IoT device.

The baseline communication sequence can therefore be represented as:

Principal → RADIUS → LDAP → RADIUS → Kerberos KDC → Gateway → IoT Device

Although this architecture already provided centralized authentication and protected command processing, it lacked an independent network-admission mechanism before the principal reached the authentication infrastructure. In other words, authentication occurred at the service level without a dedicated IEEE 802.1X control point regulating whether the requesting entity should first be admitted to the protected network. This limitation motivated the introduction of a separate **Authenticator** and IEEE 802.1X in the proposed architecture.

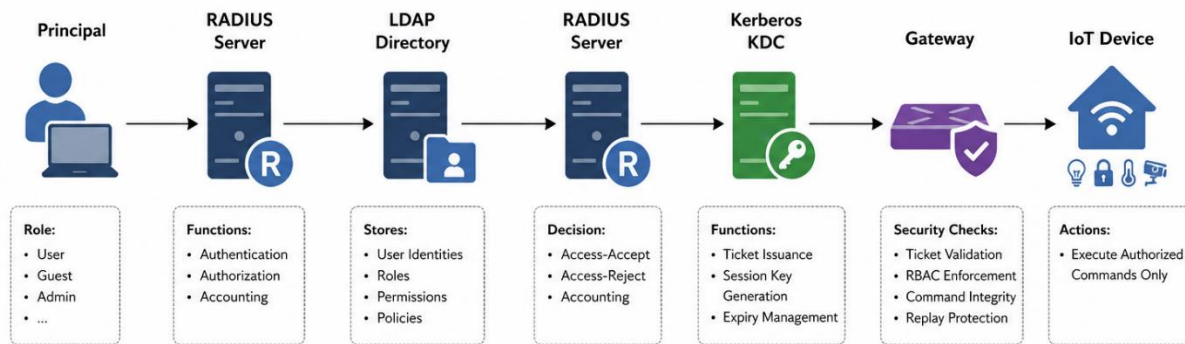


Figure 12 : Baseline Security Architecture Before IEEE 802.1X

Figure 12 : illustrates the baseline architecture in which the principal communicates directly with the RADIUS authentication infrastructure before obtaining a Kerberos-inspired ticket and accessing IoT services through the Gateway.

infrastructure, which motivated the introduction of IEEE 802.1X and a dedicated Authenticator in the proposed architecture. [2], [3]

3.3 Proposed Architecture After IEEE 802.1X

The proposed architecture enhances the baseline design by introducing **IEEE 802.1X as an independent network-admission layer** before the existing authentication, ticketing, and authorization mechanisms. IEEE 802.1X is based on three principal entities: the **Supplicant**, which requests access; the **Authenticator**, which controls network admission; and the **Authentication Server**, which validates the authentication information. EAP provides the authentication framework used within this architecture [2].

In the proposed system, the Supplicant initially remains in an **UNAUTHORIZED** state. Authentication exchanges between the Supplicant and the Authenticator are carried using **EAP over EAPOL**. The Authenticator acts as an intermediary rather than independently validating the principal's stored credentials. It encapsulates the relevant EAP authentication information within RADIUS communication and forwards it to the centralized RADIUS Authentication Server. The integration of EAP, IEEE 802.1X, and RADIUS is consistent with practical network-access architectures in which AAA functions are handled by a centralized authentication service [2], [3].

The RADIUS server then obtains the required identity information from the LDAP directory. When suitable previously retrieved information is available, the LDAP cache can reduce repeated directory lookups. LDAP-based authentication information is appropriate for caching when directory records remain relatively stable compared with their query frequency, and previous LDAP performance studies have demonstrated measurable reductions in processing latency through caching [5].

After validating the authentication request, RADIUS generates either an **Access-Accept** or **Access-Reject** response. An Access-Accept allows the Authenticator to transition the Supplicant to the **AUTHORIZED** state, whereas an Access-Reject leaves the Supplicant in the **UNAUTHORIZED** state. Consequently, only successfully authenticated principals are permitted to continue to the ticket-based service-access stage.

Following successful network admission, the authorized principal requests a temporary ticket from the **Kerberos-inspired KDC**. Kerberos-based authentication demonstrates the use of temporary service tickets and session keys to provide authenticated access to protected network services [6]. The issued ticket is then presented when the principal submits a command through the Gateway.

The Gateway acts as the primary command-level security enforcement point. Before forwarding a command to an IoT device, it verifies the ticket, applies RBAC authorization, validates the Command MAC, and checks the sequence number to determine whether the request is fresh. RBAC associates permissions with roles and allows access decisions to be made according to the

permissions assigned to each principal's role [7]. MAC verification protects the integrity of security-critical command information, while sequence-number verification prevents previously accepted requests from being reused as fresh commands.

The complete proposed workflow is therefore:

Supplicant → Authenticator → RADIUS → LDAP / LDAP Cache → Access-Accept or Access-Reject → Kerberos KDC → Gateway → RBAC → MAC Verification → Sequence Number Verification → IoT Device

This architecture establishes two distinct security boundaries. The first boundary controls **network admission** through IEEE 802.1X and centralized authentication. The second controls **service and command access** through temporary tickets, role-based authorization, integrity verification, and replay protection. Consequently, a principal must successfully satisfy multiple independent security conditions before an IoT command can reach its target device.

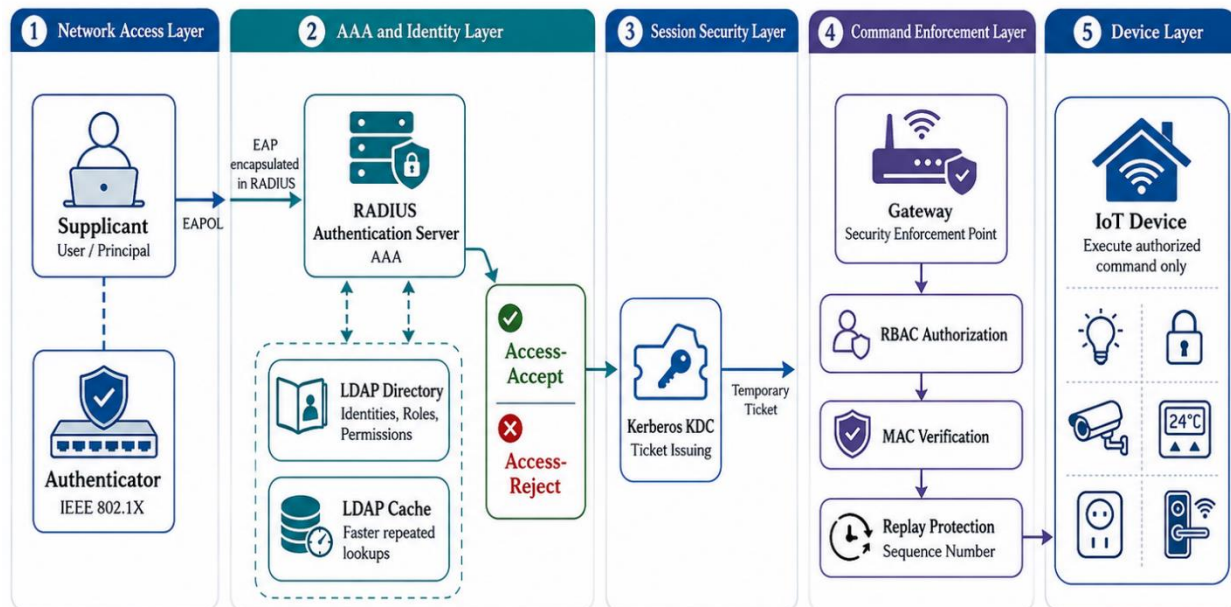


Figure 13 : Proposed Multilayer Authentication and Access Control Architecture for Smart-Home IoT

Figure 13 : presents the complete proposed architecture, beginning with IEEE 802.1X network admission and continuing through centralized authentication, ticket issuance, authorization, integrity verification, replay protection, and final IoT command execution.

3.4 System Components

The proposed architecture consists of multiple cooperating security components, with each component performing a specific function in the complete authentication and access-control process. Separating these responsibilities reduces the dependency on a single security mechanism and allows authentication, authorization, integrity, and replay protection to be independently evaluated.

3.4.1 Supplicant / Principal

The Supplicant represents the entity attempting to obtain access to the protected smart-home environment. In IEEE 802.1X, the Supplicant participates in EAP exchanges with the Authenticator and provides the authentication information required by the Authentication Server before network access can be granted [2].

Within the proposed architecture, the Supplicant is additionally associated with a principal identity that is subsequently used by the authentication, ticketing, and authorization mechanisms. Network admission does not by itself provide permission to execute arbitrary IoT commands. After admission, the principal remains subject to ticket validation and RBAC restrictions at the Gateway.

3.4.2 Authenticator

The Authenticator is introduced as an independent network-security component responsible for enforcing IEEE 802.1X access control. It mediates communication between the Supplicant and the RADIUS Authentication Server and controls whether the Supplicant remains unauthorized or is transitioned to the authorized state.

This design follows the IEEE 802.1X model in which the authentication process involves a Supplicant, an Authenticator, and an Authentication Server. EAP enables authentication information to be exchanged through these entities while centralized authentication can be provided through a RADIUS infrastructure [2].

The Authenticator therefore represents a preventive security boundary. A failed authentication process terminates network admission before the principal can continue toward the ticketing and IoT command stages.

3.4.3 RADIUS Server

The RADIUS server serves as the central AAA authentication component of the architecture. Centralized RADIUS authentication allows authentication and authorization functions to be consolidated rather than independently implemented at every network access point or protected component [3].

In this project, the RADIUS server receives the authentication information relayed by the Authenticator, obtains the required identity data through LDAP or the LDAP cache, evaluates the authentication request, and returns an acceptance or rejection decision. It therefore acts as the principal authentication decision point in the IEEE 802.1X stage.

3.4.4 LDAP Server

LDAP provides the directory layer used for centralized identity-related information. LDAP follows a client-server model in which clients retrieve structured information stored in a directory server and can be integrated with authentication systems and access-control applications [4].

Within the proposed system, LDAP stores the principal information required by the authentication and authorization architecture, including account-related information, roles, and permissions. The RADIUS server uses these directory records when evaluating authentication requests, allowing identity information to remain centrally organized rather than embedded separately into each security component.

The use of LDAP for this purpose is consistent with research showing that directory services can store authentication rules and other policy-related information [5].

3.4.5 LDAP Cache

The LDAP cache is introduced to reduce repeated directory accesses for information that has already been retrieved. Authentication and directory information is often relatively stable, making caching a useful mechanism for improving the performance of repeated LDAP operations [5].

Previous LDAP performance measurements demonstrated that entry caching can reduce back-end retrieval latency and processing time while improving throughput [5]. The reported experiment showed a reduction in processing time of approximately 40% and an increase in throughput of approximately 25% under the evaluated configuration. These values are results of the cited study and are not assumed to be identical to the results of the present project [5].

In the proposed simulation, the cache therefore provides a mechanism for comparing authentication behavior when repeated LDAP information is retrieved directly from the directory against the case in which previously available information can be served from the cache. The experimental comparison itself is presented in Chapter Four.

3.4.6 Kerberos KDC

The Kerberos-inspired KDC introduces temporary ticket-based service access after successful network authentication. Traditional Kerberos relies on a trusted authentication infrastructure, tickets, and session keys so that authenticated clients can subsequently request access to network services without repeatedly presenting their original credentials [6].

The implementation in this project is **Kerberos-inspired rather than a complete production Kerberos deployment**. Its purpose is to model the core security concept of issuing a temporary ticket after authentication and binding that ticket to information required by the Gateway to evaluate future requests.

3.4.7 Gateway

The Gateway functions as the main security enforcement point for IoT commands. It receives requests from authenticated principals and performs the remaining authorization and security checks before any request can reach a target IoT device.

The use of a gateway as an intermediate security component is particularly suitable for constrained IoT environments because complex security decisions can be placed at a more capable intermediate node rather than requiring every resource-limited device to implement the complete security architecture. IoT security literature similarly recognizes architectures in which gateways coordinate communication between IoT nodes and higher-layer services [9].

In this architecture, the Gateway performs RBAC authorization, MAC verification, and sequence-number verification. A command is forwarded only when all required checks succeed.

3.4.8 IoT Devices

The IoT devices represent the final protected resources of the proposed system. They receive only those commands that have successfully passed the preceding authentication and security layers.

This design is important because constrained IoT devices may possess limited memory, processing capability, and energy resources. Research has therefore emphasized the importance of lightweight security mechanisms and careful placement of computationally demanding security functions in IoT environments [1], [9].

Table 11 : Main Components of the Proposed Security Architecture

Main Function	Component
Initiates IEEE 802.1X/EAP authentication	Supplicant
Controls network admission	Authenticator
Performs centralized AAA authentication	RADIUS Server
Stores identity, role, and permission information	LDAP Server
Reduces repeated LDAP lookup overhead	LDAP Cache
Issues temporary access tickets	Kerberos KDC
Enforces command-level security	Gateway
Executes only accepted commands	IoT Device

3.5 Authentication Workflow

3.5.1 Initial Unauthorized State

The authentication workflow begins with the Supplicant in an unauthorized state. At this stage, the principal is not considered an authenticated member of the protected smart-home environment and cannot proceed directly to protected IoT operations.

This behavior represents the central access-control principle of IEEE 802.1X: access is conditioned on successful authentication rather than being granted immediately when a device becomes connected. Centralized server-based authentication architectures similarly prevent ordinary communication until the authentication process has successfully completed [2], [3].

3.5.2 Password Salt and Hash

The proposed authentication workflow does not conceptually depend on storing or comparing reusable plaintext passwords. Instead, password-related authentication information is represented using a salt-and-hash process, allowing credential verification to be based on derived information rather than the direct storage of the original password.

Hash-based techniques are widely used as part of authentication mechanisms, including IoT authentication systems [1].

3.5.3 Nonce and Credential Proof

A nonce is incorporated into the authentication exchange to ensure that the credential proof corresponds to the current authentication attempt. Instead of treating a previously observed authentication value as a permanently reusable credential, the proof is associated with fresh challenge information generated for the active exchange.

Within the proposed model, the nonce and credential proof therefore complement the stored credential representation. Their purpose is to allow the authentication infrastructure to validate the principal using data associated with the current authentication process rather than depending exclusively on a static transmitted value.

3.5.4 EAP over EAPOL

IEEE 802.1X uses EAP as the authentication framework, while the communication between the Supplicant and the Authenticator is carried through EAP over LAN, commonly referred to as EAPOL. The literature describes EAP as a core component of IEEE 802.1X and identifies the Supplicant, Authenticator, and Authentication Server as the major participants in the process [2].

Within the proposed implementation, EAPOL forms the first communication stage of the authentication architecture. The Supplicant exchanges the required EAP messages with the Authenticator, while direct access to the back-end authentication infrastructure is not used as the initial network admission path.

3.5.5 EAP Encapsulation inside RADIUS

After receiving the authentication information from the Supplicant, the Authenticator relays the EAP information toward the RADIUS Authentication Server. This separates the access-control role of the Authenticator from the centralized identity-validation role of the RADIUS server.

The compatibility of EAP with RADIUS is fundamental to practical IEEE 802.1X deployments. The examined 802.1X implementation literature demonstrates RADIUS-based AAA infrastructures used to manage Supplicant authentication through EAP methods [2].

3.5.6 LDAP Lookup and Cache

After receiving the authentication request, RADIUS requires identity information with which the supplied authentication proof can be evaluated. The LDAP directory serves as the authoritative directory source for this information.

When the required information is not available through the cache, an LDAP lookup is performed and the resulting information may subsequently be made available for repeated authentication requests. When an appropriate cached record is available, the system can avoid an unnecessary directory query.

Caching is especially relevant when authentication and personnel-directory information changes less frequently than it is queried. Previous LDAP research specifically identifies authentication rules as an example of relatively static directory information for which caching can improve performance [5].

3.5.7 Access-Accept and Access-Reject

After credential evaluation, the RADIUS server generates the authentication decision. A valid principal produces an Access-Accept outcome, whereas a failed validation produces an Access-Reject outcome.

The project intentionally maintains a simple binary result at this stage: a Supplicant is either accepted for network access or rejected. This prevents ambiguous intermediate access states and provides a clear input to the Authenticator's state transition.

3.5.8 Authorized and Unauthorized States

An Access-Accept response causes the Authenticator to transition the corresponding Supplicant to the **AUTHORIZED** state. Only after this transition can the principal proceed to the subsequent ticket-based and command-level security mechanisms.

Conversely, Access-Reject leaves the Supplicant in the **UNAUTHORIZED** state. The authentication flow therefore terminates before the principal is permitted to interact with the protected service-access portion of the architecture.

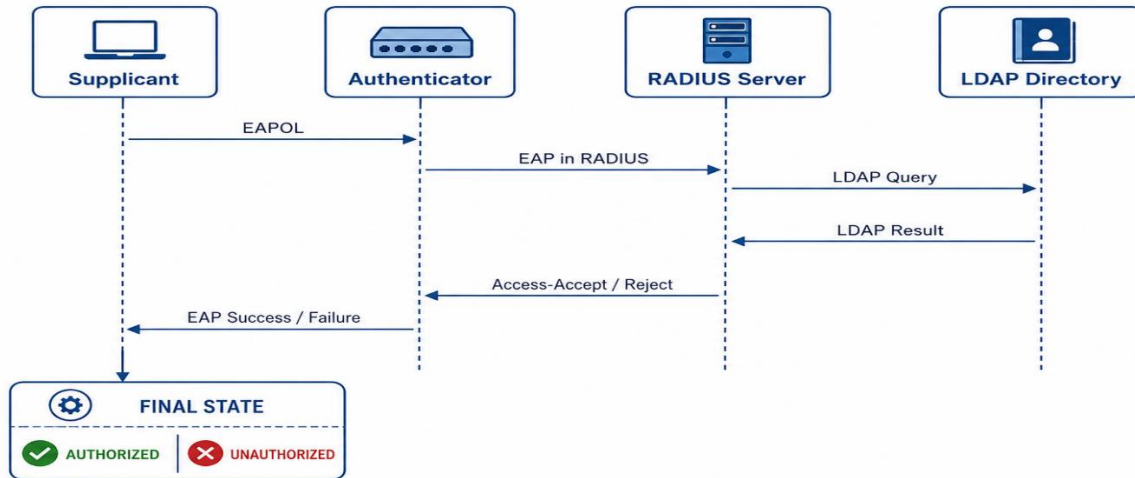


Figure 14 : IEEE 802.1X Authentication Workflow in the Proposed Architecture

Figure 14 : illustrates the IEEE 802.1X authentication workflow, where the Supplicant exchanges EAP messages through the Authenticator and RADIUS server, while LDAP provides identity information to determine the final AUTHORIZED or UNAUTHORIZED state. [2], [3]

3.6 Kerberos Ticket Workflow

3.6.1 Ticket Request

Once the principal has successfully completed IEEE 802.1X authentication and has been placed in the authorized state, the architecture proceeds to ticket-based service access. The principal requests a ticket from the Kerberos-inspired KDC.

This design follows the general principle of Kerberos in which a successfully authenticated client obtains ticket material that can later be used when requesting access to a protected service [6].

3.6.2 Session ID Validation

The ticket request is associated with the authenticated context established by the preceding security workflow. Session-related information is validated before a ticket is issued so that ticket acquisition cannot be treated as an independent path that bypasses successful authentication.

This check connects network authentication to service authorization: completion of IEEE 802.1X is a prerequisite for progressing toward ticket-based IoT access.

3.6.3 Ticket Structure

The ticket generated by the KDC contains the security context required for subsequent validation at the Gateway. Conceptually, this includes the authenticated principal identity, session-related information, authorization-related information, and temporary validity information required by the architecture.

Kerberos service tickets similarly carry identity, service, lifetime, and cryptographic information that allows the target service and the client to establish an authenticated context [6].

The project does not reproduce every field or exchange of a production Kerberos V5 deployment. Instead, it models the essential ticketing concept required for evaluating the proposed smart-home security workflow.

3.6.4 Session Key and Expiry

The issued ticket is associated with a session key used in the subsequent protected communication context. Kerberos systems similarly generate session keys for communication between authenticated principals and target services [6].

The ticket also has temporary validity rather than representing permanent authorization. Kerberos service tickets contain lifetime information and eventually require replacement when they are no longer valid [6].

3.6.5 Role and Permission Binding

The ticket also carries the authorization context required by the proposed architecture. The authenticated identity is therefore associated with its assigned role and permissions before IoT command execution.

This approach complements RBAC, in which permissions are assigned to roles and users acquire permissions through their role membership [7].

The ticket does not independently authorize every requested command. Instead, its role and permission information is subsequently evaluated by the Gateway against the requested operation.

Table 12 : Security Information Associated with the Kerberos-Inspired Ticket

Purpose	Information
Identifies the authenticated principal	Principal Identity
Associates the ticket with the authenticated context	Session Information
Represents the assigned RBAC role	Role

Defines the authorization context	Permissions
Supports protected command processing	Session Key
Limits ticket validity	Expiry

3.7 Command Security

3.7.1 `CommandRequestHeader`

After receiving a valid ticket, the principal can generate an IoT command request. The `CommandRequestHeader` represents the security-relevant information required by the Gateway to determine whether the request can be forwarded to the target device.

The request includes the information needed to identify the command and its authorization context, together with the fields used by the integrity and replay-protection mechanisms. The command is therefore treated as a structured security object rather than as an unprotected action sent directly to an IoT device.

3.7.2 Command MAC

A Message Authentication Code is used to protect the integrity and authenticity of security-sensitive command information. MAC mechanisms use keyed cryptographic processing to verify source authentication, and changes to protected message information result in a different authentication value [9].

In the proposed architecture, the Command MAC protects the command and its associated authorization-critical fields, particularly the **Role and Permission information**. The Gateway recomputes the expected MAC from the received protected information and compares it with the MAC included in the request.

If the values do not match, the request is considered to have failed integrity verification and is rejected. Research on authenticated IoT communication similarly demonstrates that message alteration results in a different MAC, allowing the receiving component to detect manipulation [9].

3.7.3 Sequence Number

A sequence number is assigned to protected command traffic to provide message freshness and enable detection of previously processed requests. The Gateway maintains sufficient state to determine whether the received sequence number represents a new valid command or an already observed value.

This mechanism is directly motivated by the replay threat. Replay attacks involve capturing legitimate network traffic and retransmitting previously observed packets at a later time [10].

A valid MAC alone cannot establish that a message is new, because a previously valid captured message may still contain a correct MAC. This observation is also recognized in message-authentication research, which notes that an attacker can retransmit a previously communicated message together with its valid MAC [9].

Therefore, MAC verification and sequence-number verification serve different purposes: the MAC verifies message integrity and authenticity, whereas the sequence number verifies freshness.

3.7.4 Gateway Security Pipeline

The Gateway executes a sequential security pipeline before forwarding an IoT command. The request must first be associated with valid authorization information. RBAC then determines whether the principal is permitted to perform the requested operation. MAC verification checks whether the security-critical request fields remain intact, and sequence-number verification determines whether the message is fresh.

The implemented decision path can be represented as:

Ticket Validation → RBAC Authorization → MAC Verification → Sequence Number Verification → IoT Device

Failure at any required security stage prevents the command from reaching the target device. This ensures that successful authentication alone is insufficient to obtain unrestricted access to smart-home resources.

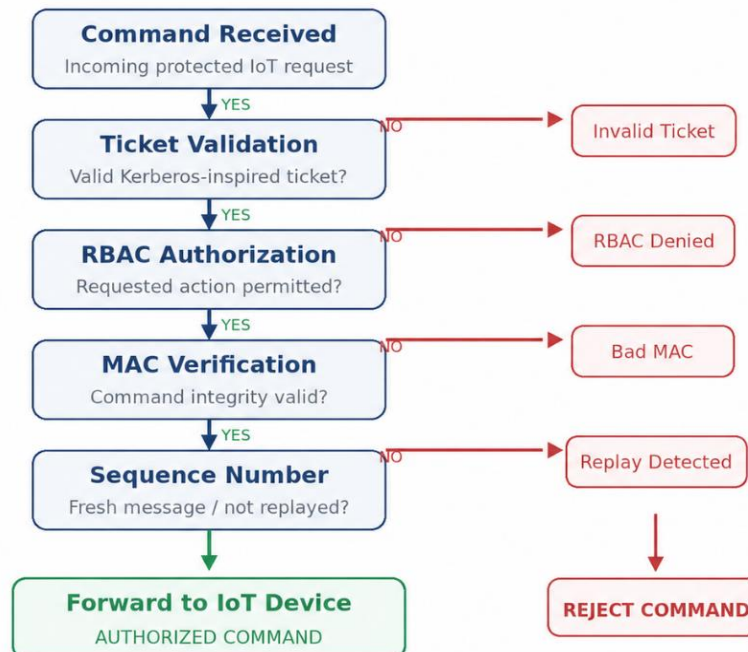


Figure 15: Gateway Security Enforcement Pipeline

Figure 15 : illustrates the Gateway Security Pipeline, where each incoming IoT command passes through sequential ticket validation, RBAC authorization, MAC verification, and sequence-number checking. A failure at any stage immediately terminates the request, while only commands satisfying all security conditions are forwarded to the target IoT device. This layered process ensures that command execution requires valid authorization, message integrity, and freshness.

3.7.5 RBAC Enforcement

RBAC provides the command-level authorization mechanism of the proposed system. Its fundamental concept is that permissions are associated with roles, while users acquire the permissions corresponding to their assigned roles [7].

This approach is suitable for the proposed smart-home environment because different users should not obtain identical authority over all connected devices. Smart-home access-control research notes that RBAC offers relatively straightforward administration and access review and may provide lightweight enforcement suitable for constrained IoT environments [7].

At the Gateway, the requested command is compared with the permissions assigned to the principal's role. A permitted operation continues through the remaining security pipeline, while a command outside the principal's permissions is rejected.

3.7.6 Device Execution

The target IoT device represents the final stage of the command-processing workflow. A command reaches this stage only when the preceding authentication, ticket, authorization, integrity, and replay-protection conditions have been satisfied.

By placing the main decision-making logic at the Gateway, the architecture avoids requiring every IoT device to independently execute the complete authentication infrastructure. This is consistent with the requirement for lightweight security in environments containing devices with limited computation and energy resources [1], [9].

3.8 Attack Scenarios

Two attack scenarios are evaluated in the proposed architecture: **Replay Attack** and **Privilege Escalation Attack**. These attacks were selected because they target two different security properties. Replay attacks challenge message freshness, whereas privilege escalation targets authorization and the integrity of privilege-related information.

3.8.1 Replay Attack

A replay attack occurs when an adversary captures legitimate communication and subsequently retransmits previously valid traffic. Consumer IoT research identifies replay attacks as a relevant threat to local smart-home communications because an adversary with access to local traffic may capture and later resend device commands [10].

In the proposed attack scenario, a legitimate command is first transmitted through the system. The attacker then attempts to resend the previously valid command. Since the captured request was originally legitimate, its identity information and MAC may still appear valid if freshness is not independently checked.

The purpose of this experiment is therefore to demonstrate the difference between command processing when replay protection is disabled and when sequence-number verification is enabled.

3.8.2 Anti-Replay Protection

Replay protection is implemented using sequence numbers. Each accepted request establishes progression in the command sequence, and a previously processed sequence value is not accepted as a fresh command.

The importance of a dedicated freshness mechanism follows from the fact that replayed traffic may preserve a valid authentication value. Message-authentication research explicitly describes replay as the retransmission of a previously communicated message that can still contain a correct MAC [9].

Therefore, the proposed architecture does not rely on Command MAC to stop replay attacks. The **MAC protects integrity**, whereas the **sequence number provides replay detection**. Separating these responsibilities is essential to the design of the security pipeline.

3.8.3 Privilege Escalation

Privilege escalation in an access-control context occurs when an entity is able to obtain or exercise permissions beyond those legitimately assigned to it. Research on IoT permission models demonstrates that over-privilege and broken access-control conditions can expose smart-home applications to unauthorized operations [8].

The proposed attack scenario focuses on unauthorized manipulation or use of privilege-related command information. An attacker attempts to cause a request to represent a role or permission level that provides greater authority than that legitimately assigned to the principal.

The security impact of such an attack could be significant in a smart-home environment because different roles may be authorized to operate different categories of devices or commands. A low-privilege principal must therefore not be able to transform an otherwise unauthorized request into an authorized operation simply by modifying privilege-related message fields.

3.8.4 Privilege Escalation Protection

The proposed architecture uses two complementary controls against privilege escalation. First, RBAC determines whether the requested command actually belongs to the set of permissions assigned to the principal's legitimate role. Role-based systems associate permissions with roles and provide a clear mechanism for determining the maximum authority assigned to a user [7].

Second, Role and Permission information is included within the data protected by the Command MAC. Consequently, unauthorized modification of these fields changes the expected MAC and causes integrity validation at the Gateway to fail. MAC-based integrity protection is specifically intended to allow the receiving entity to identify unauthorized message modification [9].

Together, RBAC and MAC verification prevent two different forms of privilege abuse. RBAC blocks commands that exceed the legitimate authorization policy, while MAC verification detects manipulation of the protected authorization information carried by the request.

Table 13: Attack Scenarios and Protocol Mechanisms

Protection	Security Property Targeted	Attack
Sequence Number	Message freshness	Replay Attack
RBAC + Command MAC	Authorization / integrity of privilege information	Privilege Escalation

3.9 Chapter Summary

This chapter presented the proposed multilayer authentication and access-control architecture for the smart-home IoT environment. The architecture extends the initial centralized authentication design by introducing IEEE 802.1X as a network admission layer before access to the remaining protected infrastructure. EAP/EAPOL is used between the Supplicant and the Authenticator, while RADIUS provides centralized authentication with identity information obtained from LDAP and, where applicable, the LDAP cache.

Following successful network authentication, the system uses a Kerberos-inspired ticketing mechanism to establish temporary authenticated service access. IoT command requests are subsequently processed by the Gateway, which performs RBAC authorization, Command MAC verification, and sequence-number-based replay detection before forwarding an accepted request to its target IoT device.

The resulting architecture therefore separates the principal security responsibilities into complementary layers. IEEE 802.1X controls network admission, RADIUS and LDAP support identity authentication, the LDAP cache addresses repeated lookup overhead, Kerberos-inspired tickets support authenticated service access, RBAC restricts commands according to roles and permissions, MAC protects the integrity of command and privilege-related information, and sequence numbers provide message freshness.

Finally, two attack scenarios were incorporated into the proposed system. Replay attacks are addressed through sequence-number verification, whereas privilege escalation is addressed through the combined enforcement of RBAC permissions and MAC-protected authorization

information. The experimental behavior and quantitative results of these mechanisms are evaluated in the following chapter.

Chapter Four

Evaluation and Results

4.1 Chapter Introduction

This chapter evaluates the proposed smart-home IoT authentication and access-control architecture implemented in ns-3.41. The evaluation focuses on verifying both the functional behavior of the security workflow and the effectiveness of the protection mechanisms introduced in the proposed system. The experiments examine IEEE 802.1X network admission, centralized authentication through RADIUS and LDAP, Kerberos-inspired ticket issuance, Gateway-based RBAC enforcement, LDAP caching, replay protection, and privilege-escalation protection.

The evaluation was organized into four main scenario families: normal system operation, LDAP cache evaluation, Replay Attack evaluation, and Privilege Escalation Attack evaluation. Each security scenario was executed independently while varying only the protection mechanism under examination, allowing the observed result to be directly associated with the corresponding security control.

The results are analyzed using authentication decisions, authorized and unauthorized states, ticket issuance, command acceptance and rejection, LDAP queries and cache hits, authentication delay, and attack-specific counters. Particular attention is given to demonstrating that successful network authentication does not automatically imply permission to execute every IoT command, since command-level authorization remains independently enforced by the Gateway.

4.2 Evaluation Methodology

The experimental evaluation was conducted using independent executions of the proposed architecture in ns-3.41. The same principal identities, device configuration, roles, permissions, and general system architecture were maintained throughout the experiments. For security comparisons, the relevant protection mechanism was alternately disabled and enabled while the remaining configuration was kept unchanged. This methodology allows the effect of each security mechanism to be evaluated independently.

The normal scenario was used to verify the expected authentication and authorization behavior without an active attack. The LDAP cache experiment compared authentication behavior with caching disabled and enabled. The Replay Attack experiment retransmitted a previously accepted command and compared the outcome with replay protection disabled and enabled. Finally, the Privilege Escalation experiment modified the role and permission claims of an authenticated but under-privileged principal and compared the outcome with privilege-escalation protection disabled and enabled.

The evaluation does not rely on a single output value. Instead, several security-related counters are observed, including the numbers of accepted and rejected authentication attempts, authorized and unauthorized principals, issued tickets, accepted and rejected commands, LDAP queries, cache hits, replay attempts, blocked replay packets, privilege-escalation attempts, and blocked forged commands.

Table 14 Evaluation Scenarios of the Proposed Architecture

Main Objective	Variable Under Evaluation	Scenario
Verify authentication and authorization behavior	No active attack	Normal Operation
Evaluate LDAP query reduction and authentication performance	Cache OFF / ON	LDAP Cache
Evaluate sequence-number-based replay protection	Protection OFF / ON	Replay Attack
Evaluate protection of role and permission claims	Protection OFF / ON	Privilege Escalation

Table 14 : summarizes the four experimental scenario families used to evaluate the proposed architecture. Each security experiment modifies the relevant protection setting while maintaining the remaining architecture unchanged, allowing the impact of each mechanism to be evaluated independently.

4.3 Evaluation Principals and Access Matrix

The evaluation environment contains six principals representing different types of users and IoT entities within the smart-home environment. Each principal is assigned a predefined role and a corresponding set of permissions. This access matrix is intentionally heterogeneous so that the architecture can be evaluated using both legitimate and unauthorized access attempts.

Ahmad represents the **OWNER** role and possesses the broadest permission set, including access to the door, light, camera, plug, and telemetry functions. Sara represents a **GUEST** and is restricted to light-control operations. Omar represents the **MAINTENANCE** role and is permitted to access plug and telemetry functions. Lina is assigned a **DISABLED** account and therefore receives no permissions. The Temperature Sensor and Motion Sensor are represented as device principals with telemetry permissions. The expected network-admission outcome is five authorized principals and one unauthorized principal.

RBAC is appropriate for this form of authorization because permissions are associated with predefined roles, and principals acquire the permissions corresponding to their assigned roles [7]. This also makes the distinction between authentication and authorization particularly important: a principal can be successfully authenticated while still being prohibited from executing a command that lies outside the permissions of its role [7].

Table 15 : Evaluation Principals and Access-Control Matrix

Expected 802.1X State	Permissions	Role	Principal
AUTHORIZED	Door, Light, Camera, Plug, Telemetry	OWNER	Ahmad
AUTHORIZED	Light	GUEST	Sara
AUTHORIZED	Plug, Telemetry	MAINTENANCE	Omar
UNAUTHORIZED	None	DISABLED	Lina
AUTHORIZED	Telemetry	DEVICE	Temperature Sensor
AUTHORIZED	Telemetry	DEVICE	Motion Sensor

Table 15 : defines the roles, permissions, and expected network-access states of the six evaluated principals. The matrix is also used by the Gateway to verify that an authenticated principal can execute only the operations permitted by its assigned role.

4.4 Normal System Results

The normal scenario was executed without enabling either of the two attack scenarios. The results confirmed that the authentication infrastructure correctly distinguished valid and invalid principals. Five authentication requests were accepted and resulted in AUTHORIZED IEEE 802.1X states, whereas one request was rejected and remained UNAUTHORIZED. Five Kerberos-inspired tickets were subsequently issued, corresponding only to successfully authorized principals.

The normal command sequence also demonstrated the expected behavior of the Gateway. Ahmad successfully executed `OPEN_DOOR` and `CAMERA_VIEW`, Sara successfully executed `LIGHT_ON`, Omar successfully executed `PLUG_ON`, and both sensor principals successfully transmitted telemetry. In contrast, Sara's attempt to execute `OPEN_DOOR` was rejected because the GUEST role does not contain the DOOR permission.

The final Gateway counters reported six accepted operations and one rejected command, with the rejection attributed to RBAC rather than authentication failure. This result is important because it demonstrates that authorization remains active after successful authentication.

```

ahmad@ubuntu: ~/ns3-dev
ahmad@ubuntu:~/ns3-dev$ ./ns3 run "scratch/iot-defense-demo.cc --simTime=15 \
--enableLdapCache=0 --enableCacheDemo=0 --enableReplayAttack=0 \
--enableReplayProtection=1 --enablePrivilegeEscalationAttack=0 \
--enablePrivilegeEscalationProtection=1 --showAuthenticationFlow=1"
[COMMAND ACCEPTED] Ahmad -> OPEN_DOOR
[DEVICE EXECUTED] Door -> OPEN_DOOR
[COMMAND ACCEPTED] Ahmad -> CAMERA_VIEW
[DEVICE EXECUTED] Camera -> CAMERA_VIEW
[COMMAND ACCEPTED] Sara -> LIGHT_ON
[DEVICE EXECUTED] Light -> LIGHT_ON
[TELEMETRY ACCEPTED] TemperatureSensor -> SEND_TEMPERATURE
[COMMAND ACCEPTED] Omar -> PLUG_ON
[DEVICE EXECUTED] Plug -> PLUG_ON
[TELEMETRY ACCEPTED] MotionSensor -> SEND_MOTION
[RBAC DENIED] Sara -> OPEN_DOOR
Reason: GUEST role does not include DOOR permission

=====
FINAL SIMULATION RESULTS
=====
[1] Authentication and IEEE 802.1X
Accepted          : 5
Rejected          : 1
Authorized        : 5
Unauthorized      : 1

[5] Gateway Security
Commands Accepted : 6
Commands Rejected : 1
Bad Command MAC   : 0
RBAC Denied       : 1

ahmad@ubuntu:~/ns3-dev$

```

Figure 16: Normal System Execution and Security Decisions

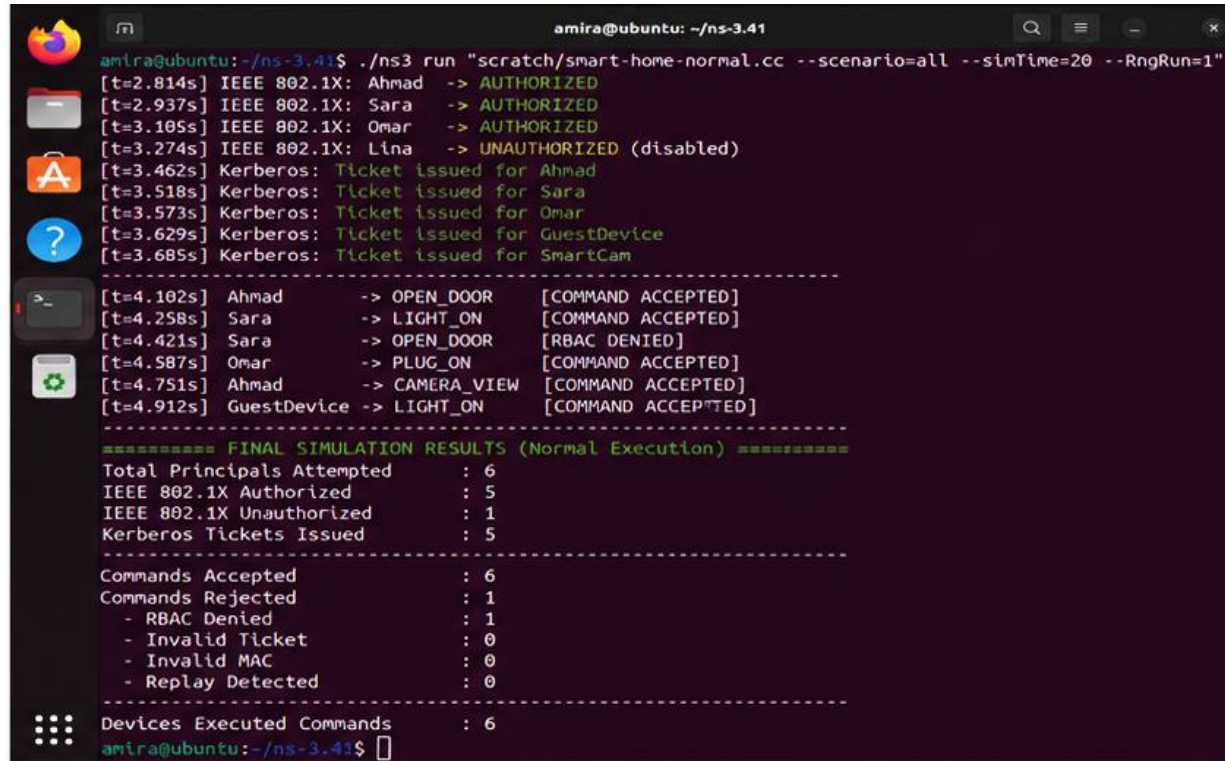
Figure 16 : shows the normal execution of authorized commands together with an RBAC rejection for an unauthorized operation. Although Sara is successfully authenticated, her OPEN_DOOR request is rejected because the GUEST role does not include the required DOOR permission.

4.5 IEEE 802.1X Results

The IEEE 802.1X evaluation confirmed that network admission is successfully separated from the subsequent command-authorization process. Ahmad, Sara, Omar, the Temperature Sensor, and the Motion Sensor received RADIUS Access-Accept decisions and were transitioned by the Authenticator to the AUTHORIZED state. Lina received an Access-Reject decision and remained UNAUTHORIZED. The final counters therefore reported five authorized principals and one unauthorized principal.

The result follows the IEEE 802.1X model in which the Authenticator controls network access while a centralized Authentication Server determines the authentication outcome [2]. Practical 802.1X infrastructures similarly use RADIUS-based AAA services to process EAP authentication and communicate the resulting access decision to the access-control component [2].

An additional observation is that only the five authorized principals received Kerberos-inspired tickets. The rejected principal did not proceed to the ticketing stage, demonstrating that successful IEEE 802.1X admission is a prerequisite for accessing subsequent protected services.



```

amira@ubuntu: ~/ns-3.41
amira@ubuntu:~/ns-3.41$ ./ns3 run "scratch/smart-home-normal.cc --scenario=all --simTime=20 --RngRun=1"
[t=2.814s] IEEE 802.1X: Ahmad -> AUTHORIZED
[t=2.937s] IEEE 802.1X: Sara -> AUTHORIZED
[t=3.105s] IEEE 802.1X: Omar -> AUTHORIZED
[t=3.274s] IEEE 802.1X: Lina -> UNAUTHORIZED (disabled)
[t=3.462s] Kerberos: Ticket issued for Ahmad
[t=3.518s] Kerberos: Ticket issued for Sara
[t=3.573s] Kerberos: Ticket issued for Omar
[t=3.629s] Kerberos: Ticket issued for GuestDevice
[t=3.685s] Kerberos: Ticket issued for SmartCam
-----
[t=4.102s] Ahmad -> OPEN_DOOR [COMMAND ACCEPTED]
[t=4.258s] Sara -> LIGHT_ON [COMMAND ACCEPTED]
[t=4.421s] Sara -> OPEN_DOOR [RBAC DENIED]
[t=4.587s] Omar -> PLUG_ON [COMMAND ACCEPTED]
[t=4.751s] Ahmad -> CAMERA_VIEW [COMMAND ACCEPTED]
[t=4.912s] GuestDevice -> LIGHT_ON [COMMAND ACCEPTED]
-----
===== FINAL SIMULATION RESULTS (Normal Execution) =====
Total Principals Attempted : 6
IEEE 802.1X Authorized : 5
IEEE 802.1X Unauthorized : 1
Kerberos Tickets Issued : 5
-----
Commands Accepted : 6
Commands Rejected : 1
- RBAC Denied : 1
- Invalid Ticket : 0
- Invalid MAC : 0
- Replay Detected : 0
-----
Devices Executed Commands : 6
amira@ubuntu:~/ns-3.41$

```

Figure 17 : IEEE 802.1X Authentication Results for Authorized and Unauthorized Principals

Figure 17 :illustrates the two possible IEEE 802.1X outcomes in the proposed system. An Access-Accept decision transitions the principal to AUTHORIZED and permits ticket issuance, whereas an Access-Reject decision maintains the principal in the UNAUTHORIZED state and terminates the authentication process.

4.6 Gateway and RBAC Results

The Gateway evaluation demonstrates that network authentication and command authorization are two independent security decisions. Successful authentication establishes that a principal is permitted to enter the protected network, whereas RBAC determines whether the same principal is permitted to execute a particular operation on a particular smart-home resource.

In the normal scenario, all commands that matched the principal's assigned permissions were accepted. Ahmad was able to access the door and camera, Sara was able to operate the light, Omar was able to operate the plug, and the two sensor principals were allowed to transmit

telemetry. However, Sara's `OPEN_DOOR` command was denied because her `GUEST` role provides only the `LIGHT` permission.

This result is consistent with the fundamental RBAC principle in which permissions are associated with roles and users obtain access through their assigned role memberships [7]. RBAC has also been identified as particularly attractive for smart-home IoT due to its relatively straightforward administration and potentially lightweight enforcement [7].

Table 16 : Normal Gateway and RBAC Command Results

Reason	Result	Command / Operation	Principal
OWNER has DOOR permission	Accepted	<code>OPEN_DOOR</code>	Ahmad
OWNER has CAMERA permission	Accepted	<code>CAMERA_VIEW</code>	Ahmad
GUEST has LIGHT permission	Accepted	<code>LIGHT_ON</code>	Sara
DEVICE has TELEMETRY permission	Accepted	<code>SEND_TEMPERATURE</code>	Temperature Sensor
MAINTENANCE has PLUG permission	Accepted	<code>PLUG_ON</code>	Omar
DEVICE has TELEMETRY permission	Accepted	<code>SEND_MOTION</code>	Motion Sensor
GUEST does not have DOOR permission	Rejected	<code>OPEN_DOOR</code>	Sara

Table 16 : confirms that command execution is determined independently from network authentication. Sara provides the clearest example: she is a valid authenticated principal, but her `OPEN_DOOR` request is rejected because the requested permission exceeds those assigned to the `GUEST` role.

4.7 LDAP Cache Evaluation

4.7.1 Without Cache

With the LDAP cache disabled, every required identity lookup must be obtained from the LDAP directory. In the evaluated cache-comparison scenario, the system generated **10 LDAP queries**, with no cache hits. The measured authentication delay was **23.852 ms**. Despite the absence of caching, the system still produced the expected security outcome: five authorized principals, five tickets, and six successful device operations.

4.7.2 With Cache

When LDAP caching was enabled, repeated identity information could be retrieved from the cache rather than requiring an additional LDAP directory query. The evaluated scenario generated **8 LDAP queries and 2 cache hits**, while the reported authentication delay decreased to **20.22 ms**. The number of authorized principals, issued tickets, and device executions remained unchanged.

The simulation trace also demonstrates the intended cache behavior directly: the first test request produced an LDAP query, whereas the following two repeated requests produced cache hits.

4.7.3 Comparison

The results demonstrate that LDAP caching improves performance without changing the security outcome. The number of LDAP queries decreased from 10 to 8, while the authentication delay decreased from 23.852 ms to 20.22 ms. At the same time, the numbers of authorized principals, tickets, and device operations remained identical in both configurations.

This trend agrees with previous LDAP performance research, which found that caching frequently requested directory entries can reduce back-end retrieval latency and processing time [5]. The cited work reported significant processing improvements under its own experimental workload; however, its numerical values should not be directly compared with those of this project because the simulation configurations and workloads differ [5].

```
=====
FINAL SIMULATION RESULTS
=====

[1] Authentication and IEEE 802.1X
Accepted      : 5
Rejected      : 1
Authorized    : 5
Unauthorized  : 1

[2] LDAP and Cache
Total Authentication Requests : 9
LDAP Queries                  : 9
Cache Hits                    : 0
Cache Misses                  : 0
LDAP Queries Saved            : 0
```

Figure 18: Authentication Results with LDAP Cache Disabled

Figure 18 : presents the authentication results when LDAP caching is disabled. In this case,

all 9 authentication requests are processed through direct LDAP lookups, resulting in 9 LDAP queries with no cache hits or saved queries.

```

File Edit View Search Terminal Help
=====
FINAL SIMULATION RESULTS
=====
[1] Authentication and IEEE 802.1X
Accepted      : 5
Rejected      : 1
Authorized    : 5
Unauthorized  : 1

[2] LDAP and Cache
Total Authentication Requests : 9
LDAP Queries                  : 7
Cache Hits                    : 2
Cache Misses                  : 7
LDAP Queries Saved            : 2
  
```

Figure 19: Authentication Results with LDAP Cache Enabled

Figure 19 : shows the authentication results when LDAP caching is enabled. The results indicate that out of 9 authentication requests, only 7 required direct LDAP queries, while 2 requests were served through cache hits, reducing the total number of LDAP queries.

Table 17 Comparison of LDAP Authentication Results with Cache Disabled and Enabled

Observation	Cache Enabled	Cache Disabled	Metric
No change	9	9	Total Authentication Requests
Decreased	7	9	LDAP Queries
Increased	2	0	Cache Hits
Increased	7	0	Cache Misses
Increased	2	0	LDAP Queries Saved
No change	5	5	Accepted Authentications
No change	1	1	Rejected Authentications

Cache Misses increased because the cache mechanism was enabled and the first-time requests were processed through cache lookup before being stored or served. This does not indicate worse performance.

4.8 Replay Attack Evaluation

Replay attacks are particularly relevant to local IoT communication because an attacker may capture legitimate network traffic and retransmit a previously valid packet at a later time [10]. In the implemented experiment, one previously accepted `OPEN_DOOR` request belonging to Ahmad is captured and retransmitted without modification. Because the packet remains unchanged, its ticket and Command MAC remain valid; the reused sequence number is therefore the property that distinguishes the replayed packet from a fresh request.

4.8.1 Protection OFF

With replay protection disabled, the duplicated sequence number was detected but the replayed packet was still allowed to continue. The replay attempt therefore resulted in a second execution of Ahmad's `OPEN_DOOR` command. The results reported one replay attempt, one detected replay, one accepted replay, and zero blocked replay packets.

Consequently, the total number of device executions increased from the normal value of six to seven because the captured door command was executed for a second time.

4.8.2 Protection ON

With replay protection enabled, the same duplicated sequence number was detected and the packet was rejected before reaching the target IoT device. The results therefore reported one replay attempt, one detection, zero accepted replay packets, and one blocked replay.

The number of device executions remained at six because the replayed `OPEN_DOOR` command did not produce an additional device action.

4.8.3 Comparison

The experiment confirms that replay **detection** and replay **enforcement** are separate aspects of the implemented mechanism. In both configurations, the reused sequence number can be identified. However, when protection is disabled the detected duplicate is allowed to proceed, whereas enabling protection causes the Gateway to reject it.

A valid MAC alone cannot solve this problem because a captured legitimate packet can still contain its original valid MAC. Message-authentication research similarly notes that replay may involve retransmission of a previously communicated message containing a correct authentication value [9]. This justifies the dedicated sequence-number freshness mechanism used in the proposed architecture.

```

Reason: GUEST role does not include DOOR permission
=====
REPLAY ATTACK
=====
Captured OPEN_DOOR packet replayed
Duplicate sequence detected
Replay packet accepted
=====
[COMMAND ACCEPTED] Ahmad -> OPEN_DOOR
[DEVICE EXECUTED] Door -> OPEN_DOOR
=====

```

Figure 20 : (a) Replay Attack Execution

The captured OPEN_DOOR command was replayed using the same sequence number. Although the duplicate sequence was detected, the replayed packet was accepted and the Smart Door executed the OPEN_DOOR command again.

```

PACKETS ISSUED : 5

[5] Gateway Security
Commands Accepted      : 7
Commands Rejected     : 1
Bad Command MAC       : 0
RBAC Denied           : 1

[6] Replay Attack
Attempts              : 1
Detected              : 1
Accepted              : 1
Blocked               : 0

```

Figure 21 : (b) Replay Attack Counters

The final results confirm one replay attempt, one detected replay, one accepted replay, and zero blocked packets. This demonstrates that detection alone does not prevent the attack when replay protection is disabled.

```

[3] RADIUS Accounting
Accounting Starts      : 5
Accounting Stops      : 5

[4] Kerberos
Tickets Issued        : 5

[5] Gateway Security
Commands Accepted     : 6
Commands Rejected     : 2
Bad Command MAC       : 0
RBAC Denied           : 1

[6] Replay Attack
Attempts              : 1
Detected              : 1
Accepted              : 0
Blocked               : 1

```

Figure 22 : Replay Attack Results with Replay Protection Enabled

Figure 22 : presents the replay attack results with replay protection enabled. The system detected the replayed packet and blocked it before it could be accepted or executed

Table 18 : Comparison of Replay Attack Results with Replay Protection Disabled and Enabled

Protection Enabled	Protection Disabled	Metric
1	1	Replay Attempts
1	1	Replay Detected
0	1	Replay Accepted
1	0	Replay Blocked
6	7	Commands Accepted
2	1	Commands Rejected
0	0	Bad Command MAC
1	1	RBAC Denied
No	Yes	Replayed Command Executed

Table 18: compares the replay attack behavior when replay protection is disabled and enabled. Although the replayed packet was detected in both cases, it was accepted and executed only when protection was disabled, whereas enabling replay protection caused the packet to be blocked before reaching the IoT device

4.9 Privilege Escalation Evaluation

The Privilege Escalation experiment evaluates whether an authenticated but under-privileged principal can gain authority beyond its legitimate role. Access-control vulnerabilities of this type

are particularly significant in IoT environments because an entity may attempt to use capabilities or permissions that were not legitimately assigned to it [8].

In the implemented attack, Sara is legitimately authenticated as **GUEST** with the **LIGHT** permission. Before transmitting a malicious `OPEN_DOOR` request, her command claims are modified from GUEST/LIGHT to **OWNER/DOOR** after the original Command MAC has already been calculated. The Kerberos-inspired ticket itself is not modified.

4.9.1 Protection OFF

With Privilege Escalation Protection disabled, the Gateway identifies the mismatch associated with the forged claims, but the protection mechanism does not enforce rejection. The forged OWNER/DOOR values therefore continue to the authorization stage and Sara's malicious `OPEN_DOOR` command is ultimately accepted and executed by the door.

The attack-specific counters report one attempt, one detected attempt, one accepted privilege escalation, and zero blocked attempts.

4.9.2 Protection ON

When Privilege Escalation Protection is enabled, the Gateway recomputes the Command MAC using the role and permission claims contained in the received request. Because the received OWNER/DOOR claims differ from Sara's original GUEST/LIGHT values over which the MAC was generated, the verification fails and the forged request is blocked before normal command authorization and device execution.

The final counters report one attack attempt, one detected attempt, zero accepted attacks, and one blocked attempt.

This behavior is consistent with the security purpose of MAC-based integrity verification: unauthorized modification of protected message information results in a different expected authentication value and can therefore be detected by the receiver [9].

4.9.3 Comparison

The results demonstrate that detecting a forged claim does not by itself provide protection unless the detection result is enforced. Both experimental configurations identify the attack attempt, but only the protection-enabled configuration prevents the forged request from reaching the target device.

The experiment also demonstrates why integrity protection and RBAC should be considered complementary mechanisms. RBAC determines whether the permission presented by a command is sufficient for the requested operation, while Command MAC verification first establishes whether those permission claims have been modified. An authorization mechanism

should not make a decision based on privilege information whose integrity has not been established.

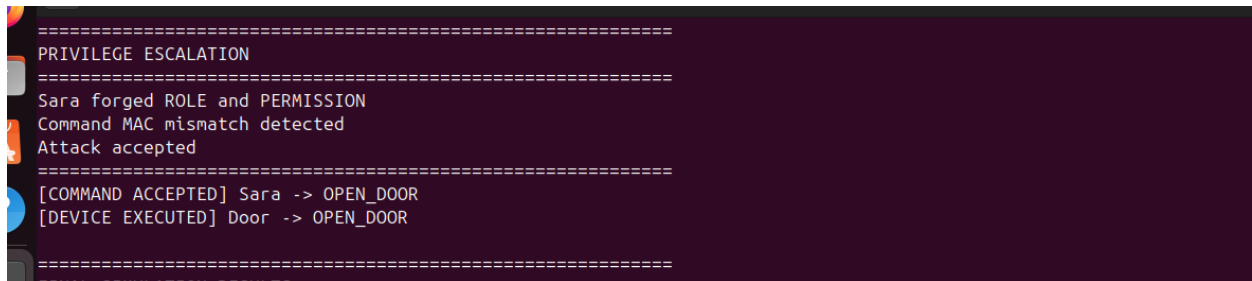


Figure 23 : Privilege Escalation Attack Execution with Protection Disabled

Figure 23 : shows that Sara forged the role and permission fields of the command request. Although the Command MAC mismatch was detected, the attack was accepted because protection was disabled, allowing the unauthorized OPEN_DOOR command to be executed.



Figure 24 : Privilege Escalation Attack Results with Protection Disabled

Figure 24 : presents the final counters for the privilege escalation scenario. One attack was attempted and detected, but it was accepted and not blocked, confirming that the forged command successfully passed through the system when protection was disabled.



Figure 25 : Privilege Escalation Attack Results with Protection Enabled

Figure 25 : shows that the privilege escalation attempt was successfully detected and blocked when protection was enabled. As a result, the forged request was not accepted, preventing the unauthorized command from being executed.

Table 19 : Comparison of Privilege Escalation Results with Protection Disabled and Enabled

Observation	Protection Enabled	Protection Disabled	Metric
No change	1	1	Attack Attempts
No change	1	1	Attacks Detected
Decreased	0	1	Attacks Accepted
Increased	1	0	Attacks Blocked
Decreased	6	7	Commands Accepted
Increased	1	0	Commands Rejected
No change	0	0	Bad Command MAC
No change	0	0	RBAC Denied
Prevented	No	Yes	Forged OPEN_DOOR Executed

Table 19 : compares the privilege escalation scenario with protection disabled and enabled. With protection disabled, the detected attack was accepted and the forged OPEN_DOOR command was executed, whereas enabling protection blocked the same attack before unauthorized device execution.

4.10 Discussion

The experimental results demonstrate that the proposed architecture does not rely on a single security decision. Instead, different mechanisms protect different stages of the smart-home communication process. IEEE 802.1X determines whether a principal is admitted to the network, RADIUS and LDAP validate identity information, Kerberos-inspired ticketing restricts subsequent service access, and the Gateway performs per-command security enforcement.

The normal scenario clearly demonstrates the distinction between **authentication and authorization**. Sara successfully completes IEEE 802.1X authentication and receives an authorized network state, yet her `OPEN_DOOR` request is independently rejected because her GUEST role does not contain the DOOR permission. This confirms that network admission does not provide unrestricted access to all smart-home resources.

The LDAP cache experiment demonstrates that performance optimization can be introduced without modifying authentication outcomes. The cache reduces LDAP queries and authentication delay while the number of authorized principals, tickets, and device operations remains unchanged. This result follows the same general performance motivation identified in LDAP research, where directory caching is used to reduce repeated retrieval overhead [5].

The two attack experiments validate the need for independent command-security mechanisms. Sequence numbers provide message freshness and prevent previously accepted packets from being reused, while Command MAC verification protects the integrity of role and permission claims. Consequently, replay protection and privilege-escalation protection address different security properties rather than duplicating the same control.

Table 20 : Overall Evaluation Summary

Main Result	Protection / Mechanism	Evaluation
5 AUTHORIZED, 1 UNAUTHORIZED	Network admission	IEEE 802.1X
Unauthorized Sara <code>OPEN_DOOR</code> rejected	Role-based command authorization	RBAC
Fewer LDAP queries and lower authentication delay	Cached identity lookup	LDAP Cache
Replay blocked when protection enabled	Sequence Number	Replay Attack
Forged privileges blocked when protection enabled	Command MAC	Privilege Escalation

Table 20 : summarizes the principal findings of the experimental evaluation. The results show that each security mechanism contributes a distinct protection property and that the combined architecture prevents unauthorized network access, unauthorized commands, replayed requests, and modified privilege claims.

4.11 Limitations

Although the evaluation demonstrates the functional effectiveness of the proposed architecture, several limitations must be acknowledged. First, the security protocols and cryptographic operations are **functionally modeled within ns-3.41 rather than implemented as full production cryptographic protocol stacks**. The objective of the simulation is therefore to evaluate the architecture, security sequence, and decision logic rather than benchmark a production deployment.

Second, the evaluated topology represents a controlled smart-home environment with a limited number of principals and IoT devices. The results demonstrate the behavior of the proposed mechanisms under the selected scenario but do not by themselves establish performance characteristics for very large smart buildings or industrial-scale IoT networks.

Finally, the security evaluation is intentionally limited to the **two implemented attack scenarios: Replay Attack and Privilege Escalation Attack**. The conclusions concerning attack prevention therefore apply specifically to the implemented threat model and should not be interpreted as exhaustive protection against every possible IoT security threat. The project documentation likewise defines these as the two implemented attack scenarios.

4.12 Conclusion

This chapter evaluated the proposed authentication and access-control architecture through normal operation, LDAP cache comparison, Replay Attack testing, and Privilege Escalation testing. The results confirmed that IEEE 802.1X successfully separates authorized and unauthorized principals, while Kerberos-inspired tickets are issued only after successful network authentication.

The Gateway correctly enforced command-level RBAC permissions, demonstrating that successful authentication does not automatically grant unrestricted device access. LDAP caching reduced directory queries and authentication delay without altering security decisions. In addition, sequence-number verification prevented replayed commands from executing, while Command MAC verification prevented modified role and permission claims from being used for privilege escalation.

Overall, the evaluation demonstrates that the proposed layered architecture provides complementary controls for network admission, identity authentication, service access, authorization, message integrity, and message freshness.

4.13 Future Work

The proposed architecture can be extended in several directions. Future evaluation can increase the number of principals, smart-home devices, and concurrent authentication requests to investigate scalability in larger smart-home and smart-building environments.

Additional EAP methods, particularly **EAP-TLS** and **EAP-PEAP**, can also be evaluated within the IEEE 802.1X authentication stage. Another important extension is replacing the lightweight functional cryptographic models used in the current ns-3.41 implementation with full cryptographic protocol implementations.

Finally, the proposed architecture can be adapted and evaluated in more complex IoT domains, including **Industrial IoT environments**, where authentication, authorization, and command-integrity requirements may involve substantially larger infrastructures. These extensions are consistent with the identified future directions of the project.